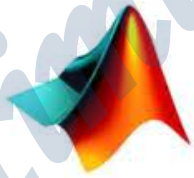
Scientific Computing (MATH 594)

A Concise Note

Tutor: Dr. Jeevan Kafle
Lecturer at
The Central Department of Mathematics
Institute of Science and Technology
Tribhuvan University
Kathmandu, Nepal



MATLAB

Timilsina RC

Prepared By
Ramchandra Timilsina
E-mail: timilsinarc2053@gmail.com
Roll No.:42/076
Kaski, Nepal

January 17, 2023

CONTENTS

1 Introduction To Matlab **1**

1.1 Features of Matlab 1
1.2 Matlab Environment 2
1.3 Rules for Naming File 3
Exercise 4

2 BASIC OPERATION **5**

2.1 Arithmetic Operation 5
2.2 Special Variables and Constants 5
2.3 Elementary Function 6
2.4 Relational Operation 7
2.5 Logical Operation 7
2.6 Vector Function 7
2.7 Matrix Functions 8
2.8 Addressing Matrix or Vector Components 9
2.9 Array Manipulation and Appending 9
2.10 Meaning of Predefined Variables and Operators 10
2.11 Input and Output Commands 10
2.12 The M-files 12
2.12.1 Script Files 12
2.12.2 Function Files 14

Emilsina RC

2.12.3 Anonymous Function 16
Exercise 18

3 Plotting **19**
3.1 2-D Plot 19
3.1.1 Line Specifiers: 19
3.2 Legends 20
3.3 Multiple Plots 25
3.4 Sub-plots 27
3.5 3-D Plots 29
Exercise 31

4 Branches and Loops **32**
4.1 if ... end statement 32
4.2 if...else...end statement 32
4.3 if... elseif...elseif...else...end statements 33
4.4 nested if statements 33
4.5 switch-case statement 35
4.6 for loop 36
4.7 while loop 37
4.8 break statement 38
4.9 continue statement 39
Exercise 40

5 Numerical Integration **42**
5.1 Trapezoidal Rule 42
5.2 Simpson's 1/3-Rule 45
5.2.1 Single Application 45
5.2.2 Composite Application 46
5.3 Simpson's 3/8-Rule 49
5.3.1 Single Application 49
5.3.2 Composite Application 50

Imilsina RC

Exercise 53

6 Numerical Solution of IVPs in ODEs 54

6.1 Forward (explicit) Euler Method 54
6.2 Backward (implicit) Euler Method 56
6.3 2nd Order Runge-Kutta Method 57
6.4 4th Order Runge-Kutta Method 59
Exercise 62

7 Debugging 63

7.1 Debugging 63
7.2 Errors in Matlab 63
7.3 Possible Issues for Debugging 63
7.4 Minimizing Bugs 64
Exercise 64

8 Numerical Methods for System of Linear Equation 65

8.1 System of Linear equations 65
8.2 Crammer's Rule 66
8.3 Gaussian Elimination Method 66
8.4 Gauss Jordan Method 68
8.5 LU Factorization Method 69
8.6 Gauss Seidel Method 70

9 Root of Non-linear Equation 74

9.1 Bisection Method 74
9.2 Newton-Rapshon Method 75
Exercise 76

10 Numerical Solution to the PDE 77

10.1 Finite Difference Method 77
10.2 Heat Equation 77
10.3 Wave Equation 82

Timilsina RC

10.4 Laplace Equation 85
10.5 Finite Element Method 87

11 title **88**
Exercise 88

Timilsina RC

CHAPTER 1

INTRODUCTION TO MATLAB

MATLAB (Matrix Laboratory) is a fourth-generation high level programming language and has interactive environment for numerical computation, visualization and programming. It is developed by MathWorks.
It allows matrix manipulations, plotting of functions and data, implementation of algorithms, creation of user interfaces, create models and application, etc.

Timilsina RC

1.1 Features of Matlab

- It is a high-level language for numerical computation, visualization and application development.
- It provides interactive environment for iterative exploration, design and problem solving.
- It provides vast library of mathematical functions such as linear algebra, statistics, Fourier analysis, optimization, numerical integration and ordinary differential equation.
- It provides built-in graphics for visualizing data and tools for creating custom plots.
- It provides tools for building application with custom graphical interfaces.
- There are many inbuilt tools like analyzer and debugger for analysis and debugging of codes written in Matlab.

1.2 Matlab Environment

- **Current folder:** This panel allows us to access the project folders and files.
- **Command window:** This is the main area where commands can be entered at the command line. It is indicated by the command prompt (»).
- **Work space:** The workspace shows all the variables created or imported from files.
- **Command history:** The panel shows or rerun commands that are entered at the command line.
- **Edit window:** It permits a user to create and modify Matlab programs by creating new M files or to modify existing ones.
- **Current directory window:** It shows the path of the current directory.
- **Figure window:** It displays plots and graphs.
- **Help files:** A user can get help from Matlab through Matlab documentation.

Timisina RC

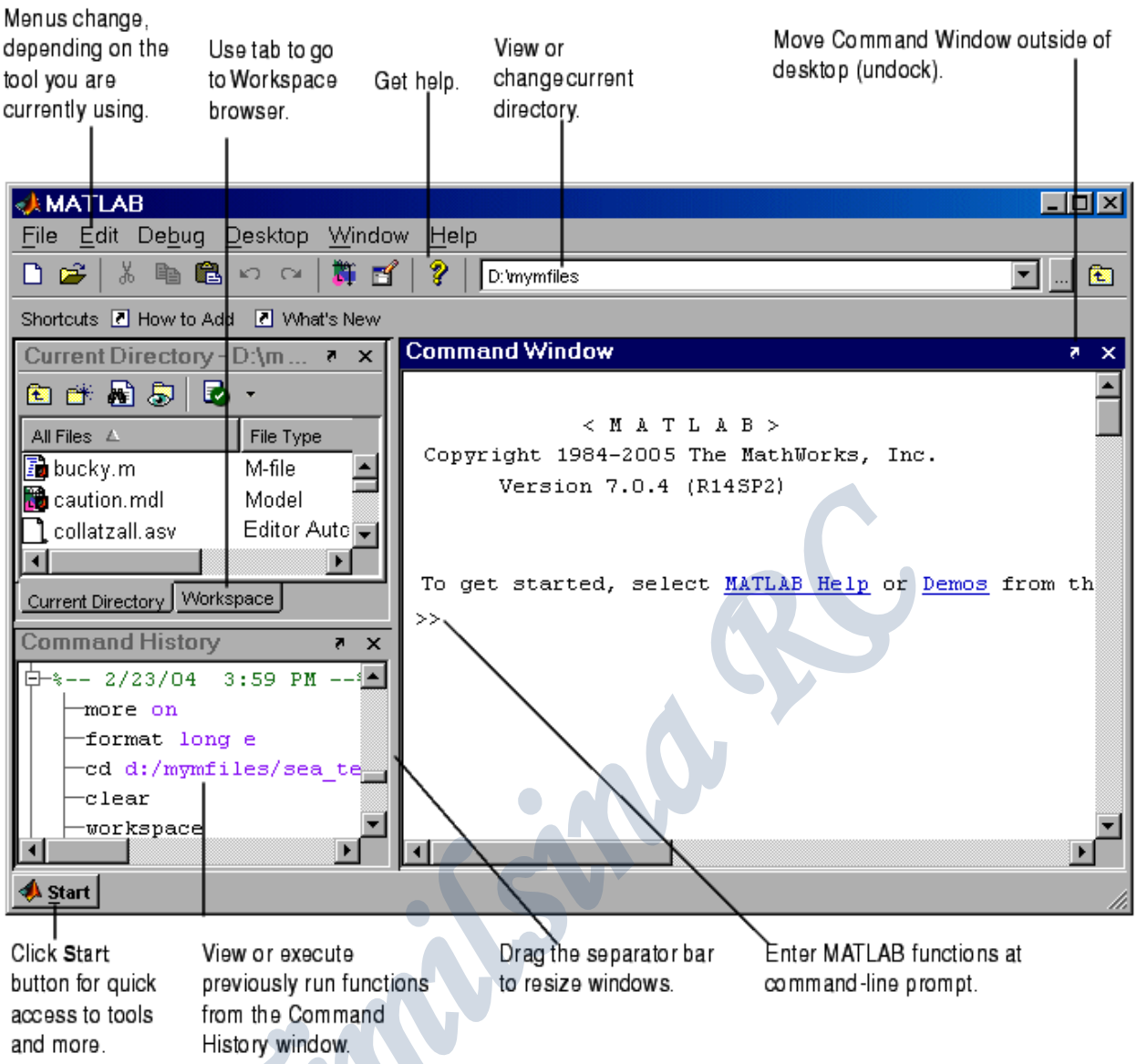


Figure 1.1: The graphical interface to the Matlab workspace [1].

Tanisha RC

1.3 Rules for Naming File

- First letter should not be number. e.g 1plot.m
- First letter should not be symbol. e.g *plot.m. But underscore sign (_) is allowed.
- First letter should not be space. i.e all file name (variables) start with a letter.
- Build-in function such as det, exp, sqrt , etc are not allowed.

EXERCISE

1. Discuss different types of windows use in the MATLAB. Write their uses.

Timilsina RC

CHAPTER 2

BASIC OPERATION

2.1 Arithmetic Operation

Operator	Purpose
+	Addition
-	Subtraction
*	Scalar and matrix multiplication operator
.*	Array multiplication operator
^	Power
.^	Element wise power
\	Left-division (e.g. $6\backslash 3 = 0.5$)
/	Right-division (e.g. $6/3 = 2$)
.\	Element wise left-division
./	Element wise right-division

Timisina RC

2.2 Special Variables and Constants

Name	Meaning
%	Adding comments
ans	Most recent answer
eps	Accuracy of floating-point precision (Very very small positive number)

i,j	The imaginary unit $\sqrt{-1}$
inf	Infinity
pi	The number π
NaN	Undefined numerical result (not a number)
who	Displays all the used variables
whose	Displays little more about the variables
what	Lists all Matlab files in the current directory
clear or clc	To clear the command window
clear all	To deletes all variables from the memory
clear (x)	Only deletes variable x
Ctrl + C	To break execution and get new prompt (»)
exist	Checks for existence of file or variable
global	Declares variables to be global
help	Searches for a help topic
quit	Stops Matlab
cd	Changes current directory
date	Displays current date
delete	Deletes a file

2.3 Elementary Function

sin()	Sine	abs()	Absolute value
cos()	Cosine	sign()	Signum function
tan()	Tangent	factorial()	Factorial functio
asin()	Inverse sine	angle()	Phase angle
acos()	Inverse cosine	ceil()	Ceiling function
atan()	Inverse tangent	floor()	Floor function
sinh()	Hyperbolic sine	fix()	Round towards zero
asinh()	Inverse hyperbolic sine	round()	Rounded to nearest integer
exp()	Exponential	mod()	Modulus after division

Imtisha RC

sqrt()	Square root	rem()	Reminder after division
log()	Natural logarithm	conj()	Complex conjugate
log10()	logarithm base 10	real()	Real part
log2()	logarithm base 2	imag()	Imaginary part

2.4 Relational Operation

Operation	Meaning
<	Less than
<=	Less than or equal
>	Greater than
>=	Greater than or equal
==	Equal
~=	Not equal

2.5 Logical Operation

Operation	Meaning
&	And
&&	And with shortcut evaluation
|	Or
||	Or with shortcut evaluation
Xor	Exclusive or
~	Not

2.6 Vector Function

Commands	Meaning

Imalsina RC

length()	Length of a vector
max()	Maximum value
min()	Minimum value
sum()	Sum of components of vector
sort()	Sorting of vector elements
norm()	Norm of vector
prod()	Product of components
mean()	Mean
median()	Median
std()	Standard deviation
any()	Any non-zero value
all()	All non-zero value
dot(A,B)	Dot product of vectors A and B
cross(A,B)	Cross product of vectors A and B

2.7 Matrix Functions

Commands	Meaning	Commands	Meaning
size()	Size of matrix	A'	Transpose of matrix
rank()	Rank of matrix	diag()	Diagonal element
inv()	Inverse	rand()	Uniform random numbers
det()	Determinant	randn()	Normal random numbers
norm()	Norm	randperm()	Random permutation
cond()	Condition number	flipud()	Flips from up to down
eig()	Eigen values	fliprl	Flip from left to right
hess()	Hessenberg matrix	rot90()	Rotates 90°
zeros(m,n)	Matrix of 0's	circshift()	Circularly shifts the matrix
ones(m,n)	Matrix of 1's	triu()	Upper triangular matrix
eye(n)	Identity matrix	tril()	Lower triangular matrix

Emulsina RC

rref()	Reduced row echelon form	pinv	Pseudoinverse of a matrix

2.8 Addressing Matrix or Vector Components

Commands	Meaning
$V(n)$	n^{th} element of vector
$V(n_1 : n_2)$	n_1^{th} to n_2^{th} elements of vector
$V([n_1, n_2, \dots])$	n_1^{th} , n_2^{th}, ... elements of vector
A(m,n)	Value of mth row and n^{th} column of matrix
$A(m_1 : m_2, n)$	Sub-matrix of m_1^{th} to m_2^{th} row and n^{th} column
$A(m, n_1 : n_2)$	Sub-matrix of mth row and n_1^{th} to n_2^{th} column
$A(m_1 : m_2, n_1 : n_2)$	Sub-array of m_1^{th} to m_2^{th} row and n_1^{th} to n_2^{th} column
$A(m, :)$	Sub-array of mth row and all columns
$A(:, n)$	Sub-array of all rows and n^{th} column
$A([m_1, m_2, \dots], n)$	Sub-matrix of $m_1^{th}, m_2^{th}, \dots$ row and n^{th} column
$A(m, [n_1, n_2, \dots])$	Sub-matrix of mth row and $n_1^{th}, n_2^{th}, \dots$ column
$A([m_1, m_2, \dots], [n_1, n_2, \dots])$	Sub-matrix of $m_1^{th}, m_2^{th}, \dots$ row and $n_1^{th}, n_2^{th}, \dots$ column
$A(n)$	n^{th} elements of A column-wise

2.9 Array Manipulation and Appending

Commands	Meaning
$V(n) = [\,]$	Deleting n^{th} element of vector
$V(n) = p$	Replacing n^{th} element of vector by p
$A(m, n) = p$	Replacing m^{th} row n^{th} column element of matrix by p
$A(m_1 : m_2, n_1 : n_2) = [M]$	Replacing sub-array of m_1^{th} to m_2^{th} row and n_1^{th} to n_2^{th} column by matrix M

Timilsina RC

$A(:,n)=[\,]$	Deleting n^{th} column
$A(n)=p$	Replacing n^{th} element of A column-wise by p

2.10 Meaning of Predefined Variables and Operators

Commands	Meaning
comma(,) or space bar	To separate the elements of row e.g » A=[a,b,c] or [a b c]
semi-colon(;) or enter	To separate rows e.g » B=[a,b,c;1,2,3] or [a b c ; 1 2 3]
semi-colon(;)	Not to display the answer e.g » A=[a,b,c];
colon(:)	1. For an increment in a vector 2. To denote all columns or rows in a matrix
a:b	Generates a vector from a to b with default increment one e.g » x=2:5 $\Rightarrow$ 2 3 4 5
a:s:b	Generates a vector with increment s e.g » y= 0:2:10 $\Rightarrow$ 0 2 4 6 8 10
linspace(a,b)	Generates linearly spaced vector of length 100 (default value)
linspace(a,b,n)	Generates linearly spaced vector of length n e.g » z=linspace(0,15,5)$\Rightarrow$ 0 3 6 9 12 15
logspace(a,b,n)	Generates logarithmically spaced vector of length n from 10^a to 10^b
find(A>n)	Finds all entries greater than n of vector or matrix

2.11 Input and Output Commands

Command	Purpose
input()	Displays prompts (») and waits for input

Tanisha RC

disp()	Displays contents of an array or string
fprintf()	Performs formatted writes to screen or file
format()	Controls screen-display format
fscanf()	Read formatted data from a file

The **fscanf** and **fprintf** commands behave like C **scanf** and **printf** functions. They support the following format codes:

Format Code	Purpose
% s	Format as a string
% d	Format as an integer
% f	Format as a floating point value
% e	Format as a floating point value in scientific notation
% g	Format in the most compact form: %f or %e.
\ n	Insert a new line in the output string
\ t	Insert a tab in the output string
%8.2f	floating point format in a field eight characters wide, including two digits after the decimal point

The format function has the following forms used for numeric display:

Format Function	Display upto
format short	Four decimal digits (default)
format long	16 decimal digits
format short e	Five digits plus exponent
format long e	16 digits plus exponents
format bank	Two decimal digits
format +	Positive, negative, or zero
format rat	Rational approximation
format compact	Suppresses some line feeds
format loose	Resets to less compact display mode

Amisha RC

2.12 The M-files

Matlab allows writing two kinds of program files:

2.12.1 Script Files

- A script file is a user created file with a sequence of commands in it. It is equivalent to typing all the commands stored in the script file, one by one, at the Matlab prompt in the command window.
- The file is saved with a **.m** extension to its name.
- The file name must begin with a letter and the rest of the character may include digits and underscores but not period other than the last one **.m** .
- The script file can be executed by typing its name (without the **.m** extension) in the command window.
- Script files work on global variable, that is variables currently present in the workspace.
- Scripts do not accept inputs and do not return any outputs.
- Script files are useful when we have to repeat a set of commands several times.

Tamilsina RC

Example. 2.1: Solution of quadratic equation (Script file)

Question: Write a program that asks the user for the value of a,b,c then identify nature of root and value of roots of the quadratic equation $ax^2 + bx + c = 0$.

Matlab Code:

quad_script.m*

```
% To find roots of quadratic equation ax^2+bx+c=0:
a = input('Enter non-zero value of a: ');
b = input('Enter value of b: ');
c = input('Enter value of c: ');
k = b^2-4*a*c;
x1 = (-b+sqrt(k))/(2*a);
x2 = (-b-sqrt(k))/(2*a);
if k < 0
   x = real(x1);
   y = imag(x1);
   fprintf('The imaginary roots are %d \x00B1 i %d \n',x,y)
   % \x00B1 for plush minus sign
else
fprintf('The real roots are x1=%d and x2=%d \n' ,x1,x2);
end
```

Output:

```
>> quad_script
Enter non-zero value of a: 1
Enter value of b: 2
Enter value of c: 3
The imaginary roots are -1 ± i 1.414214e+00
>> quad_script
Enter non-zero value of a: 1
Enter value of b: 4
Enter value of c: 3
The real roots are x1=-1 and x2=-3
```

Tamilsina RC

2.12.2 Function Files

- If we want to do a big set of computations but in the end we want only a few outputs, a script file is not the right choice. What we need in this case is a function file.
- A function file is also an M-file, like a script file, except it has a function definition line on the top that defines the input and output explicitly.
- The function file must be saved in your required folder with the file name **fname.m**.
- The function file can be executed by giving the values of the input arguments by typing the format of function file in the command window.
- Functions files can accept inputs and return outputs.
- Internal variables are local to the function.

Syntax:

1. function[outarg1, outarg2,...] = fname (inarg1, inarg2,...)
2. function [output] = fname (inarg1, inarg2,...)
3. function output = fname (inarg1, inarg2,...)
4. function [ ] = fname (inarg1, inarg2,...)
5. **function fname (inarg1, inarg2,...)**

Imdsina RC

Example. 2.2: Solution of quadratic equation (Function file)

quad_script.m | quad.m*

```
1   % To find roots of quadratic equation ax^2+bx+c=0:
2   function quad(a,b,c)
3   % We must save the file with name same as function name
4   k = b^2-4*a*c;
5   x1 = (-b+sqrt(k))/(2*a);
6   x2 = (-b-sqrt(k))/(2*a);
7   if k < 0
8      x = real(x1);
9      y = imag(x1);
10     fprintf('The imaginary roots are %d \x00B1 i %d \n',x,y)
11     % \x00B1 for plush minus sign
12  else
13  fprintf('The real roots are x1=%d and x2=%d \n' ,x1,x2);
14  end
```

Output:

```
>> quad(1,2,3)
The imaginary roots are -1 ± i 1.414214e+00
>> quad(1,4,3)
The real roots are x1=-1 and x2=-3
```

Timilsina RC

Example. 2.3: Perpendicular distance of a point from plane

Question: The distance d from a point (x_o, y_o, z_o) to a plane $Ax + By + Cz + D = 0$ is given by $d = \frac{|Ax_o+By_o+Cz_o+D|}{\sqrt{A^2+B^2+C^2}}$. Write a Matlab program to find d for various inputs parameters A, B, C, D, x_o, y_o & z_o.

Matlab Code:

Editor - D:\RC_MatLab\pdist.m

quad.m | pdist.m

```
1     % To find the perpendicular distance of (2,1,5) from plane -2x+3y+z+3=0.
2     function pdist(A,B,C,D,xo,yo,zo)
3 —       d = (abs(A*xo+B*yo+C*zo+D)/sqrt(A^2+B^2+C^2));
4 —       fprintf('Required perpendicular distance is d= %f \n' ,d);
```

Command Window

New to MATLAB? See resources for Getting Started.

```
>> pdist(-2,3,1,3,2,1,5)
Required perpendicular distance is d= 1.870829
fx >>
```

Tamilsina RC

2.12.3 Anonymous Function

An anonymous function is like an inline function in traditional programming languages, defined within a single Matlab statement. It consists of a single Matlab expression and any number of input and output arguments.

We can define an anonymous function right at the Matlab command line or within a function or script.

```
f = @(arglist) expression
```

Example. 2.4: Valve of exponential function

Matlab Code:

```
» f = @(x) exp(x)
» A = [0:5]
» f(A)
```

Output:

```
A = 0    1    2    3    4    5
ans= 1.0000    2.7183    7.3891    20.0855 54.5982    148.4132
```

Example. 2.5: Power of a number

Matlab Code:

```
» power = @(x,n) x.^n
» A = [0:5]
» power(A,2)
```

Output:

```
A = 0    1    2    3    4    5
ans= 0    1    4    9    16    25
```

Timilsina RC

EXERCISE

1. Write a program that asks the user for the value of a, b, c then identify nature of root and value of roots of the quadratic equation $ax^2 + bx + c = 0$.

2. Write the short notes on script files and function files in MATLAB programming.

3. How do you distinguish the script file and function files in MATLAB programming? Also, describe details about arrays in MATLAB programming with examples.

4. Write a program to find the solution of the system of equations x-2y+3z=-1, 4x+5y+6z=-1, 7x+8y+10z=0.

5. Celsius temperatures can be converted to Fahrenheit by multiplying by 9, dividing by 5, and adding 32. Assign a variable called C the value 37, and implement this formula to assign a variable F the Fahrenheit equivalent of 37 Celsius. Write program in Matlab code of this problem.

6. Let C be a 3×3 matrix. Write a Matlab command to replace its second row by the third row.

7. Write a Matlab command to generate a matrix $A = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 \end{pmatrix}$ using matrix functions eye( ), ones( ), zeros( ).

8. Write the meaning of the following functions of built in Matlab commands: ceil(n), prod(x), sign(n) , where n is a number and x is a vector

Timilsina RC

CHAPTER 3

PLOTTING

3.1 2-D Plot

plot (X, Y, 'line specifiers')

OR

fplot('function', limits, 'line specifiers')
e.g fplot('x*cos(6x)',[-2,4],':*r')

3.1.1 Line Specifiers:

Line Style	Specifire	Line Color	Specifire	Marker Type	Specifire
Solid	-	Red	r	Plush sign	+
Dotted	:	Green	g	Circle	o
Dashed	–	Blue	b	Asterisk	*
Dash-dot	-.	Cyan	c	Point	.
		Magenta	m	Square	s
		Yellow	y	Diamond	d
		Black	k	Triangle	^

Tanisha RC

3.2 Legends

Commands	Purpose
xlabel ('string')	Label along x-axis
ylabel ('string')	Label along y-axis
title ('string')	Put title on the graph
axis([xmin xmax ymin ymax])	Minimum and maximum limits of axes
axis equal	For equal axes
axis square	For square axes
grid on	Grid lines on the graph
legend('string1','string2')	Indexing the graph
gtext('string')	Enables label placement by mouse
text(x,y,'string')	Enables label at (x,y)

Note

- **Circle at center (0,0):** x = r* cos(A), y= r*sin(A)
- **Circle at center (h,k):** x = h + r* cos(A), y= k + r*sin(A)
- **ellipse with semi-major axis (a) and semi-minor axis(b) at center (0,0):** x = a* cos(A), y= b*sin(A)
- **ellipse with semi-major axis (a) and semi-minor axis(b) at center (h,k):** x = h + a* cos(A), y= k + b*sin(A)
- **Parabola** $y^2 = 4ax$**:** x = a*t^2, y=2*a*t .
- **Hyperbola:** x = a*sech(A), y = b*tanh(A)

Tamilsna RC

Example. 3.1: Circle:

Matlab Code:

Editor - D:\RC_MatLab\circle.m*

circle.m* | Circle2.m | Ellipse.m | hyp.m | Hyperbola.m

```
% Plotting circle of desired radius:
function circle(r)
A=0:0.01:2*pi;
x=r*cos(A);    % written in parametric form.
y=r*sin(A);
plot(x,y,'r-.','linewidth',4)
axis('equal')
axis([-6 6 -6 6])
xlabel('x-axis')
ylabel('Y-axis')
text(0,0,'. (0,0)')
title('The circle of radius r')
legend('Circle')
```

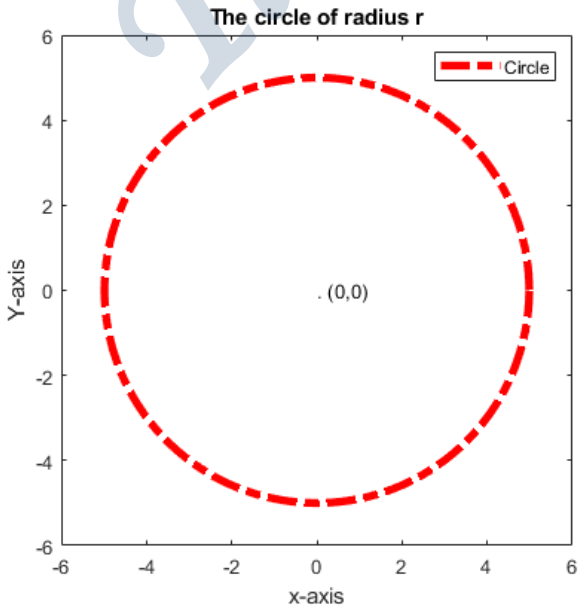
Command Window

New to MATLAB? See resources for Getting Started.

```
>> circle(5)
```

Imilsina RC

Output:



Example. 3.2: Ellipse:

Question: Write the program that asks the user for semi- major axis, semi-minor axis and center of ellipse and draw a ellipse in central form by using different legends.

Matlab Code:

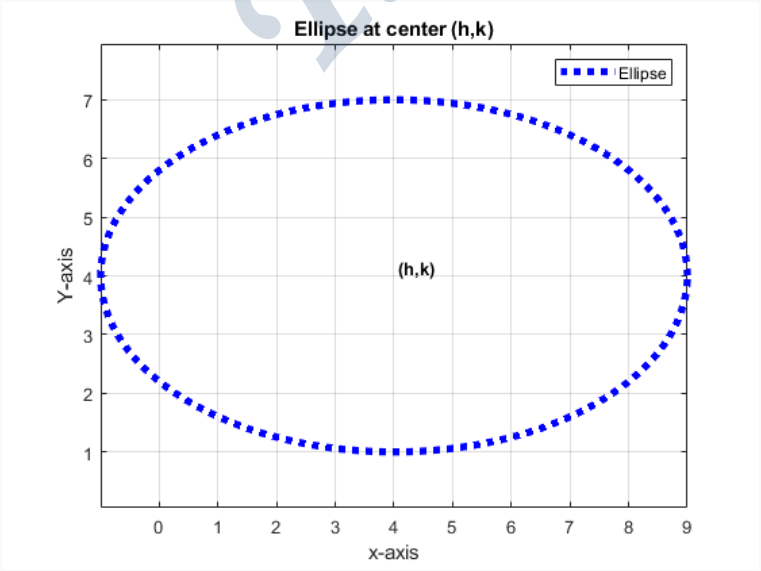
Editor - D:\RC_MatLab\Ellipse.m*

Ellipse.m*

```
% Plotting ellipse:
A=0:0.01:2*pi;
a=input('Enter the value of semi-major axis: \n');
b=input('Enter the value of semi-minor axis: \n ');
h=input('Enter x-cordinate of center: \n');
k=input('Enter y-cordinate of center: \n ');
x=h+a*cos(A);
y=k+b*sin(A);
plot(x,y,'b:','linewidth',4)
axis equal
grid on
xlabel('x-axis')
ylabel('Y-axis')
gtext('\bf (h,k)')   % \bf -> Bold \it->italic
title('Ellipse at center (h,k)')
legend('Ellipse')
```

Tanisha RC

Output: Ellipse with (a,b)=(5,3) and (h,k)=(4,4).



Example. 3.3: Parabola:

Matlab Code:

hyp.m | Hyperbola.m | Parabola.m | Simple_plot.m

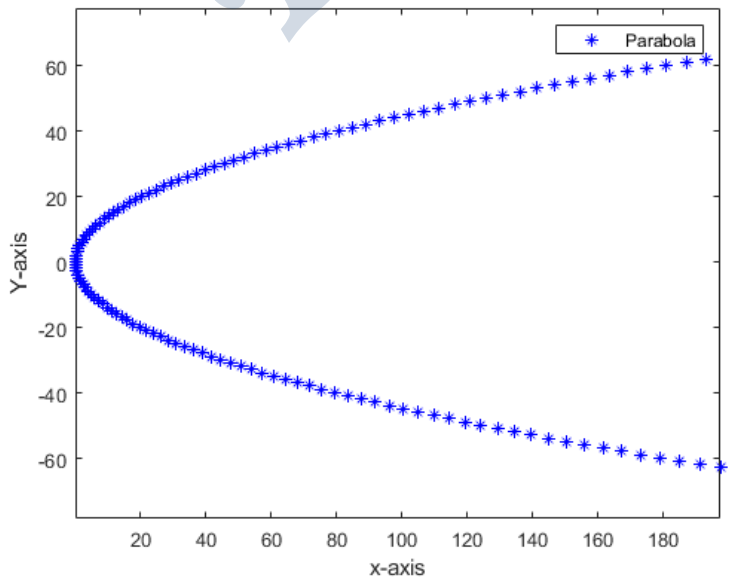
```
1   % Plotting parabola y^2 = 4ax :
2   t=-2*pi:0.1:2*pi;
3   a=input('Enter the value of a: ');
4   x=a*t.^2;
5   y=2*a.*t;
6   plot(x,y,'*b');
7   axis('equal');
8   xlabel('x-axis');
9   ylabel('Y-axis');
10  legend('Parabola');
11
```

Command Window

New to MATLAB? See resources for Getting Started.

```
>> Parabola
Enter the value of a: 5
```

Output:



Timisha RC

Example. 3.4: Hyperbola:

Matlab Code:

Editor - D:\RC_MatLab\Hyperbola.m

hyp.m | Hyperbola.m | Simple_plot.m

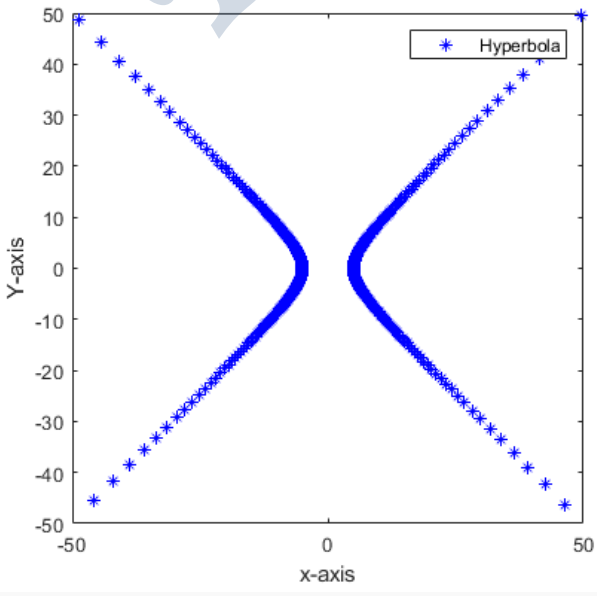
```
% Plotting hyperbola :
theta=0:0.01:2*pi;
a=input('Enter the value of a: ');
b=input('Enter the value of b: ');
x=a*sec(theta);
y=b*tan(theta);
plot(x,y,'*b');
axis('equal');
axis([-50 50 -50 50])
xlabel('x-axis');
ylabel('Y-axis');
```

Command Window

New to MATLAB? See resources for Getting Started.

```
Enter the value of b: 5
>> Hyperbola
Enter the value of a: 5
Enter the value of b: 5
```

Output:



Tinilsha RC

3.3 Multiple Plots

plot(x,y,'s',u,v,'s',t,h,'s')
» y versus x, v versus u, and h versus t with line specifiers 's'.

OR

```
» plot(x,y,'s')
» hold on
» plot(u,v,'s')
» plot(t,h,'s')
» hold off
```

Timilsina RC

Example. 3.5: Comparing function and its derivative:

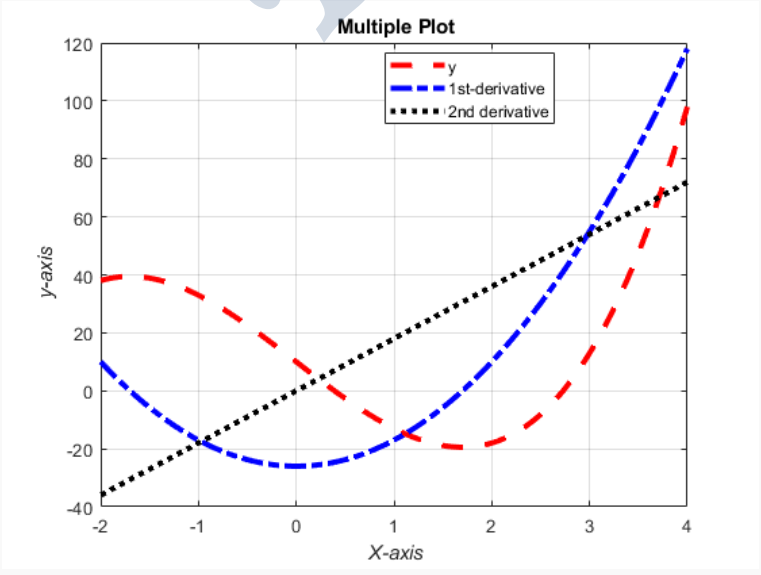
Question: Plot of the function $y = 3x^3 - 26x + 10$ and its first and second derivatives for all $-2 \leq x \leq 4$ in the same plot.

Matlab Code:

Multiple_plot.m* | Multiple_plot2.m

```
x = -2:0.01:4;
y = 3*x.^3-26*x+10;
y1 = 9*x.^2-26;
y2 = 18*x;
plot(x,y,'r--','linewidth',3)
hold on
plot(x,y1,'b-.','linewidth',3)
plot(x,y2,'k:','linewidth',3)
hold off
% OR plot(x,y,'r--',x,y1,'b-.',x,y2,'k:','linewidth',3)
title('\bf Multiple Plot')
xlabel('\it X-axis')
ylabel('\it y-axis')
grid on
legend('y','1st-derivative','2nd derivative')
```

Output:



Tmisina RC

3.4 Sub-plots

When we create an array of plots in the same figure, each of these plots is called a subplot. The **subplot** command is used for creating subplots.

subplot(m,n,p), plot(x,y,'s')
Where,
m = number of rows
n = number of columns
p = Specification for a particular plot
i.e

1	4
2	**5**
3	6

→ **subplot(3,2,5)**

Timilsina RC

Example. 3.6: Subplot:

Matlab Code:

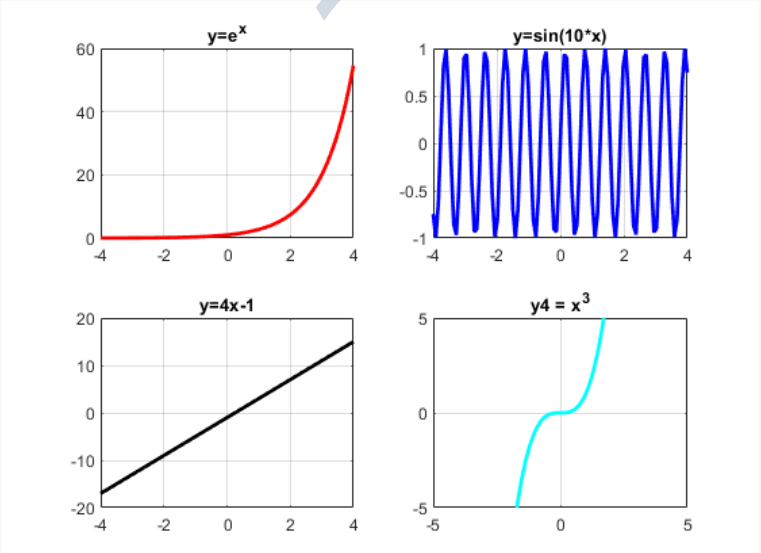
Editor - D:\RC_MatLab\sub_plot.m

Multiple_plot2.m | sub_plot.m

```
x = linspace(-4,4,100);
y1 = exp(x);
y2 = sin(10*x);
y3 = 4*x-1;
y4 = x.^3;
subplot(2,2,1),plot(x,y1,'r-','linewidth',2)
title('y=e^x')
grid on
subplot(2,2,2),plot(x,y2,'b-','linewidth',2)
title('y=sin(10*x)')
grid on
subplot(2,2,3),plot(x,y3,'k-','linewidth',2)
title('y=4x-1')
grid on
subplot(2,2,4),plot(x,y4,'c-','linewidth',2)
title('y4 = x^3')
axis([-5 5 -5 5])
grid on
```

Imtiaz RC

Output:



3.5 3-D Plots

1. **plot3 (x,y,z):** 3-D plot.
2. **surf (x,y,z):** Surface plot.
3. **surfc (x,y,z):** Surface + contour plot.
4. **surfl (x,y,z):** Surface plot with lighting.
5. **contour (x,y,z):** 2-D contour plot.
6. **contour3 (x,y,z):** 3-D contour plot.
7. **contour (x,y,z,c):** Contour with c number of curves.
8. **contourf (x,y,z,c):** Contour with color fill.
9. **mesh (x,y,z):** Mesh plot.
10. **meshz (x,y,z):** Mesh curtain plot.
11. **meshc (x,y,z):** Mesh + contour plot.
12. **waterfall (x,y,z):** Water fall plot

Tamilsina RC

Example. 3.7: 3-D Plots:

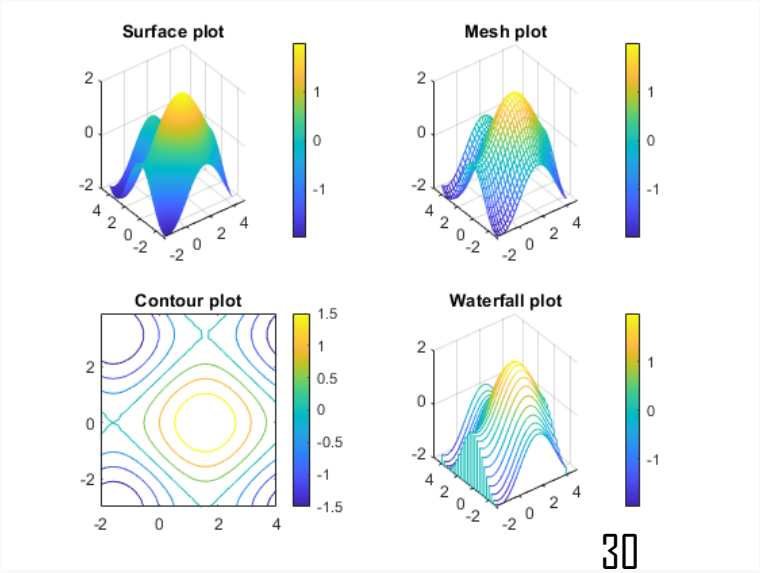
Editor - D:\RC_MatLab\threeD_plot.m

threeD_plot.m | meshplot.m

```
x = -2:0.2:4;
y = -3:0.3:4;
[u,v] = meshgrid(x,y);          % to make grid
z = sin(u) + cos(v);
subplot(2,2,1), surf(u,v,z)
title('Surface plot')
shading interp
colorbar
subplot(2,2,2), mesh(u,v,z)
title('Mesh plot')
shading flat
colorbar
subplot(2,2,3), contour(u,v,z)
title('Contour plot')
shading interp
colorbar
subplot(2,2,4), waterfall(u,v,z)
title('Waterfall plot')
shading interp
colorbar
```

Tamilsina RC

Output:



EXERCISE

1. Write a program that asks the user for semi-major axis and semi-minor axis and center of ellipse and then draw a ellipse in central form by using different legends.

2. Write a program to compute and plot the following quantities for t runs from 1 to 10:

   (i) $x = t \ \sin t$

   (ii) $z = \frac{\sin t^2}{t^2}$

   (iii) $y = \sin t \cos t$

   (iv) $\frac{1}{10}(t - \frac{t^{3/2}}{10})^2$

3. Write a program by the following plot and fplot commands plots the graph of the function $f(x) = e^{0.1} \sin x$ for x between 0 and 20.

4. Write Matlab program to plot a function $y = e^x, 0 \leq x \leq 5$ with title 'Exponential Plot', X-label 'Time' and Y-label 'Population'.

5. Plot the following functions in same figure window using subplot:
   $y = x^2 + 5$, $y = x^2 - 3x + 7$; $-6 \leq x \leq 8$

Timisina RC

CHAPTER 4

BRANCHES AND LOOPS

4.1 if ... end statement

An **if ... end** statement consists of a boolean expression followed by one or more statements. If the expression evaluates to true, then the block of code inside the if statement will be executed. If the expression evaluates to false, then the first set of code after the end statement will be executed.

```
» if <condition>
» statement(s);
» end
```

4.2 if...else...end statement

If the first condition is not true then we can define other statements to run by using the **else** keyword.

```
» if <condition>
» statement(s);
» else
» statement(s);
» end
```

Timilsina RC

Example. 4.1: Printing Even Numbers

Fact.m | even.m*

```
% even number from 1 to 10
for i = 1 : 10
    if mod(i,2) == 0
        disp(i)
    end
end
```

Output:

» 2 4 6 8 10

4.3 if... elseif...elseif...else...end statements

An **if statement** can be followed by one (or more) optional **elseif...** and an **else statement**.

```
» if <condition 1>
» statement(s);
» elseif <condition 2>
» statement(s);
:
» elseif <condition n>
» statement(s);
» else
» statement(s);
» end
```

4.4 nested if statements

We can use one **if or elseif statement** inside another **if or elseif statement(s)**.

Timilsina PC

Example. 4.2: Grade System

grade.m*

```
% Internal Assesment marks : 20
% Final marks : 30
A = input('Internal assesment obtained marks: \n');
D = input('Final marks: \n');
fprintf('Revised internal marks: \n');
P = min(A,D/30*20*1.2)
fprintf('Total obtained marks: \n');
B = P+D
fprintf('Grade ')
if A > 20 || D > 30
   disp('Wrong Information')
else
   if B >= 45
      disp('A')
   elseif B >= 40
      disp('A-')
   elseif B >= 35
      disp('B+')
   elseif B >= 30
      disp('B')
   elseif B >= 25
      disp('B-')
   else
      disp('Fail')
   end
end
```

Tmilsina RC

Output:

» grade
Internal assessment obtained marks : 20
Final marks: 22
Revised internal marks:
P = 17.6000
Total obtained marks:
B = 39.6000
Grade B+

4.5 switch-case statement

A switch block conditionally executes one set of statements from several choices. Each choice is covered by a case statement. An evaluated switch expression is a scalar or string. When a case is true, matlab executes the corresponding statements and then exits the switch block. The otherwise block is optional and executes only when no case is true.

```
» switch <expression>
» case <expression>
» statements
» case <expression>
» statements
...
» otherwise
» statements
» end
```

Timisina RC

Example. 4.3:

Switch.m

```
grade = input('Enter your grade : ');
switch grade
   case 'A'
      fprintf('Excellent! \n');
   case 'B'
      fprintf('Well done! \n');
   case 'C'
      fprintf('Very good! \n');
   case 'D'
      fprintf('You passed! \n');
   case 'F'
      fprintf('Better try again! \n');
   otherwise
      fprintf('Invalid grade \n');
end
```

Command Window

New to MATLAB? See resources for Getting Started.

```
>> Switch
Enter your grade : 'C'
Very good!
```

Timisina RC

4.6 for loop

A **for loop** is used to repeat a statements or group of statements for a fixed number of times.

```
» for <index=values>
» statements
» end
```

Example. 4.4: Factorial of Natural Numbers

Editor - D:\RC_MatLab\factorial.m

Fact.m | factorial.m

```
% To find n!
n = input('Enter the value of n : ');
if n < 0
   disp('Factorial is undefined for negative numbers')
elseif n == 0
    fact = 1;
    fprintf(' %d ! = %d  \n', n, fact)
else
     fact = 1;
     for i = 1 : n
        fact = fact*i;
     end
     fprintf(" %d ! = %d  \n", n, fact)
end
```

Command Window

New to MATLAB? See resources for Getting Started.

```
>> factorial
Enter the value of n : 5
 5 ! = 120
```

Tinku Sina RC

4.7 while loop

The **while loop** is used to execute a statement or group of statements for an indefinite number of times until the condition is not satisfied.

```
» while <expression>
» statements
» end
```

Example. 4.5: Factorial of Natural Numbers

Fact.m* factorial.m

```
% To find n!
n = input('Enter the value of n : ');
if n < 0
   disp('Factorial is undefined for negative numbers')
elseif n == 0   % (< , ==, <=, >=, ~ =)
    fact = 1;
    fprintf(' %d ! = %d  \n', n, fact)
else
    fact = 1;
    i = 1 ;
    while i <= n     % There should not be gap between < and =
      fact = fact*i ;
      i = i + 1 ;
    end
    fprintf(" %d ! = %d  \n", n, fact)
end
```

Command Window

New to MATLAB? See resources for Getting Started.

```
Enter the value of n : 5
 5 ! = 120
```

Tanisha RC

4.8 break statement

The break statement terminates execution of for or while loop. Statements in the loop that appear after the break statement are not executed.

Example. 4.6:

```
for i = 1 : 10
    if i == 5
        break
    end
    disp(i)
end
```

Output:

```
»    1    2    3    4
```

4.9 continue statement

The continue statement is used for passing control to next iteration of for or while loop. The continue statement in Matlab works somewhat like the break statement. Instead of forcing termination, however, 'continue' forces the next iteration of the loop to take place, skipping any code in between.

Example. 4.7:

```
for i = 1 : 10
    if i == 5
        continue
    end
    disp(i)
end
```

Output:

```
»    1    2    3    4    6    7    8    9    10
```

Tanisha RC

EXERCISE

1. Print multiplication table from 1 to 10 by using Matlab code.

Matlab Code:

Prod.m

```
% Multiplication table for 1 to 10
for i =1 : 10
    for j = 1 : 10
        P = i * j ;
        fprintf('%d * %d = %d \n', i, j, P)
    end
end
```

Output:

» 1 * 1 = 1
1 * 2 = 2
...
10 * 10 = 100

2. Write a program code to calculate n factorial for positive value of n by using (i) for loop and (ii) while loop.
   Solution: Already done in examples .

3. Write short notes on break and continue statements in loop.

4. Write a program to determine the given number is even or odd.

5. Suppose you are distributing letter grades using the following criteria: greater than 80 : A, greater than 70: B, greater than 60: C, greater than 50: D and less than 50: F. Write a program that will assign the grades for given obtained mark.

6. Distinguish between control statements and conditional statements. Also, describe about if, else, elseif statements with examples.

Tamilsina RC

7. Write a program that asks the user for an age and then classifies the age according to the following scheme: Error < 0 <= Baby < 1 <= Child < 13 <= Teenager < 18 <= Adult < 60 <= Senior < 120 <= Error .

8. Write a program to find the sum of $1^2 + 2^2 + 3^2 + \cdots + 100^2$.

Timilsina RC

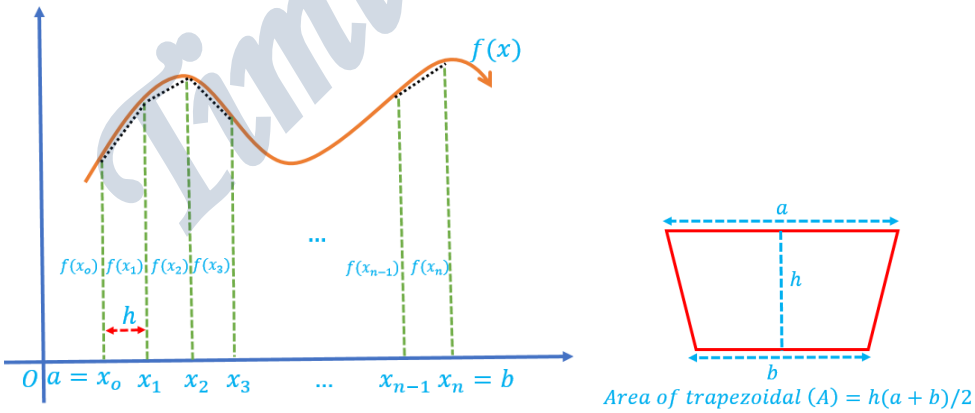
CHAPTER 5

NUMERICAL INTEGRATION

Some times it is difficult to find exact integration of function. In such case we have to find approximated value. The method of finding approximated value of definite integral is called numerical integration. There are various methods some of them are described below.

5.1 Trapezoidal Rule

Let $y = f(x)$ be a continuous curve defined on $[a, b]$. Let us partition the interval $[a, b]$ into n equal parts of size h as shown in figure below.



Where, $x_1 = a + h, x_2 = a + 2h, x_3 = a + 3h, \ldots, x_n = a + nh$. Therefore, $h = \frac{b-a}{n}$, n = number of partition.

Now, the total area enclosed by the curve is sum of area of each n trapezoidals. This can be written as

Timilsina RC

$$I = \frac{h}{2}[f(x_o) + f(x_1)] + \frac{h}{2}[f(x_1) + f(x_2)] + \ldots + \frac{h}{2}[f(x_{n-1}) + f(x_n)]$$

$$\begin{aligned} I &= h\{\frac{1}{2}[f(a) + f(b)] + f(x_1) + f(x_2) + \cdots + f(x_{n-1})\} \\ &= h[S + \sum_{i=1}^{n-1} f(x_i)] \\ &\quad Where\ S = \frac{1}{2}[f(a) + f(b)] \end{aligned}$$

Example. 5.1:

Question: Find the value of integral $\int_0^{\pi/2} x \sin x \ dx$ by using **Trapezoidal rule** and compare with exact solution. Also make a program.

Matlab Code:

Editor - D:\RC_MatLab\traP_rule2.m

traP_rule2.m | Trap_rule.m

```
% Trapezoidal Rule
f =  @(x) x*sin(x);
a = 0;
b = pi/2;
n = 5;
h = (b-a)/n;
s = 0.5*(f(a)+f(b));
for i = 1:n-1
    s = s+f(a+i*h);
end
I = h*s;
fprintf('Approximate area is : %d \n',I)
```

Command Window

New to MATLAB? See resources for Getting Started.

```
>> traP_rule2
Approximate area is : 1.008265e+00
```

Tanisha RC

Exact Solution:

$$\begin{aligned} I &= \int_0^{\pi/2} x \sin x \, dx \\ &= -x \cos x\big|_o^{\pi/2} + \int_0^{\pi/2} \cos x \\ &= 0 + \sin x\big|_o^{\pi/2} \\ &= \sin\frac{\pi}{2} - \sin 0 \\ &= 1 - 0 \\ &= 1 \end{aligned}$$

Trapezoidal Rule:

Here, $f(x) = x \sin x$, $a = 0, b = \pi/2$. Choose $n = 5$ then $h = \frac{b-a}{n} = \frac{\frac{\pi}{2}-0}{5} = \frac{\pi}{10}$.

So, $x_0 = a = 0$

$x_1 = a + h = \frac{\pi}{10}$

$x_2 = a + 2h = \frac{\pi}{5}$

$x_3 = a + 3h = \frac{3\pi}{10}$

$x_4 = a + 4h = \frac{2\pi}{5}$

$x_5 = a + 5h = \frac{\pi}{2}$

Then,

x	$f(x) = x \sin x$
$x_0 = 0$	$f(x_0) = 0$
$x_1 = \frac{\pi}{10}$	$f(x_1) = 0.0970$
$x_2 = \frac{\pi}{5}$	$f(x_1) = 0.3693$
$x_3 = \frac{3\pi}{10}$	$f(x_1) = 0.7624$
$x_4 = \frac{2\pi}{5}$	$f(x_1) = 1.1951$
$x_5 = \frac{\pi}{2}$	$f(x_1) = \frac{\pi}{2}$

Timilsina RC

Now,

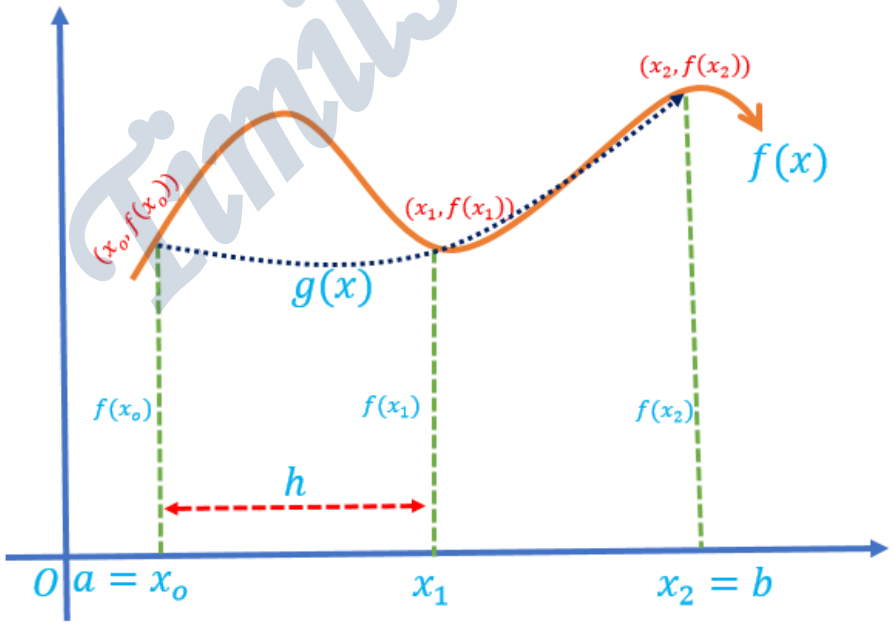
$$\begin{aligned} I &= h\{\frac{1}{2}[f(x_0)+f(x_5)]+f(x_1)+f(x_2)+f(x_3)+f(x_4)\} \\ &= \frac{\pi}{10}\{\frac{1}{2}[0+\frac{\pi}{2}]+0.0970+0.3693+0.7624+1.195\} \\ &= 1.0083 \end{aligned}$$

Error (E) = $\frac{|Exact-Approx.|}{Exact} \times 100\ \% = \frac{|1-1.0083|}{1} \times 100\ \% = 0.83\ \%$

5.2 Simpson's 1/3-Rule

5.2.1 Single Application

Let $y = f(x)$ be a continuous curve defined on $[a, b]$. Let us partition the interval $[a, b]$ into 2 equal parts of size $h = \frac{b-a}{2}$ as shown in figure below.



Here we approximate $f(x)$ by the **quadratic** polynomial $g(x)$ passing through the points $(x_o, f(x_o))$, $(x_1, f(x_1))$ and $(x_2, f(x_2))$ which is obtained by using **Lagrange's Interpolation Method** as follows .

Timilsina RC

$$g(x) = \frac{(x-x_1)(x-x_2)}{(x_o-x_1)(x_o-x_2)}f(x_o) + \frac{(x-x_o)(x-x_2)}{(x_1-x_o)(x_1-x_2)}f(x_1) + \frac{(x-x_o)(x-x_1)}{(x_2-x_o)(x_2-x_1)}f(x_2)$$

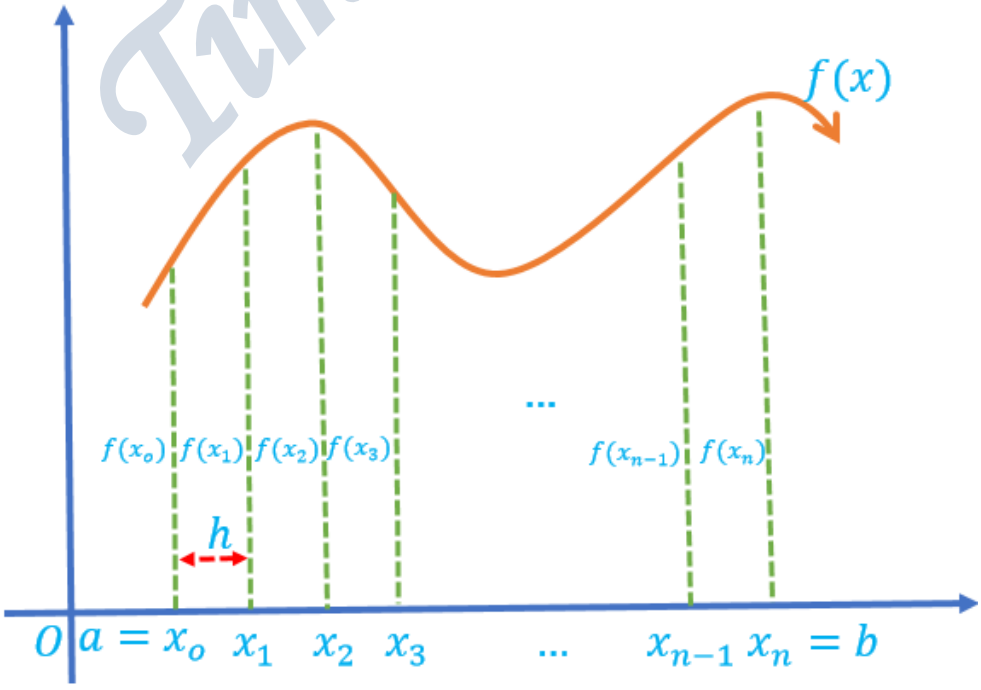
By some algebraic manipulation , we will get $g(x)$ is in the form $a_o x^2 + a_1 x + a_2$ with $f(x_i) = g(x_i)\ ; i = 0, 1, 2$ and $f(x) \approx g(x)\ \forall x \in [a, b]$. Then,

$$\begin{aligned} I &= \int_a^b f(x)dx \\ &\approx \int_a^b g(x)dx \\ &= \frac{b-a}{6}[f(x_o) + 4f(x_1) + f(x_2)] \\ I &\approx \frac{h}{3}[f(x_o) + 4f(x_1) + f(x_2)] \end{aligned}$$

is the required formula for Simpson's 1/3 rule for single application.

5.2.2 Composite Application

Let $y = f(x)$ be a continuous curve defined on $[a, b]$. Let us partition the interval $[a, b]$ into n equal parts of size h as shown in figure below.



Timilsina RC

Where, $x_1 = a + h, x_2 = a + 2h, x_3 = a + 3h, \ldots, x_n = a + nh$. Therefore, $h = \frac{b-a}{n}$, n = number of partition.
Now,

$$\begin{aligned} I &= \int_a^b f(x)dx \\ &= \int_{x_o=a}^{x_2} f(x)dx + \int_{x_2}^{x_4} f(x)dx + \ldots + \int_{x_{n-2}}^{x_n=b} f(x)dx \\ &\approx \frac{h}{3}[f(x_o) + 4f(x_1) + f(x_2)] + \frac{h}{3}[f(x_2) + 4f(x_3) + f(x_4)] + \ldots + \\ &\quad \frac{h}{3}[f(x_{n-2}) + 4f(x_{n-1}) + f(x_n)] \\ &\quad \because \textit{Using formula of single application.} \\ &= \frac{h}{3}[\{f(x_o) + f(x_n)\} + 4\{f(x_1) + f(x_3) + \cdots + f(x_{n-1})\} \\ &\quad +2\{f(x_2) + f(x_4) + \cdots + f(x_{n-2})\}] \\ I &\approx \frac{h}{3}[\{f(a) + f(b)\} + 4\sum_{1,3,5,\ldots}^{n-1} f(x_i) + 2\sum_{2,4,6,\ldots}^{n-2} f(x_i)] \end{aligned}$$

is the required formula for Simpson's 1/3 rule for composite application.

$$\begin{aligned} I &= \frac{h}{3}[f(x_o) + 4f(x_1) + f(x_2)] \quad (\textit{Single Application}) \\ I &= \frac{h}{3}[\{f(x_o) + f(x_n)\} + 4\{f(x_1) + f(x_3) + \cdots + f(x_{n-1})\} \\ &\quad +2\{f(x_2) + f(x_4) + \cdots + f(x_{n-2})\}] \ (\textit{Composite Application}) \\ &= \frac{h}{3}[\{f(a) + f(b)\} + 4\sum_{1,3,5,\ldots}^{n-1} f(x_i) + 2\sum_{2,4,6,\ldots}^{n-2} f(x_i)] \end{aligned}$$

Timsina RC

Example. 5.2:

Question: Find the value of integral $\int_0^{\pi/2} x \sin x \; dx$ by using **Simpson's 1/3-Rule** and compare with exact solution. Also make a program.

Matlab Code:

Editor - D:\RC_MatLab\OneThird_rule.m

RK2ndmethod.m | OneThird_rule.m

```
1     % 1/3 Rule
2     f = @(x) x*sin(x);
3     a = 0;        % lower limit
4     b = pi/2;     %upper limit
5     n = 6 ;       %multiple of 2
6     h = (b-a)/n ;  % step size
7     s = f(a)+f(b);
8     for i = 1:2:n-1
9         s = s+4*f(a+i*h);
10    end
11    for i = 2:2:n-2
12        s = s+2*f(a+i*h);
13    end
14    I = (h/3)*s;
15    fprintf('Approximated area is %d \n', I);
```

Command Window

New to MATLAB? See resources for Getting Started.

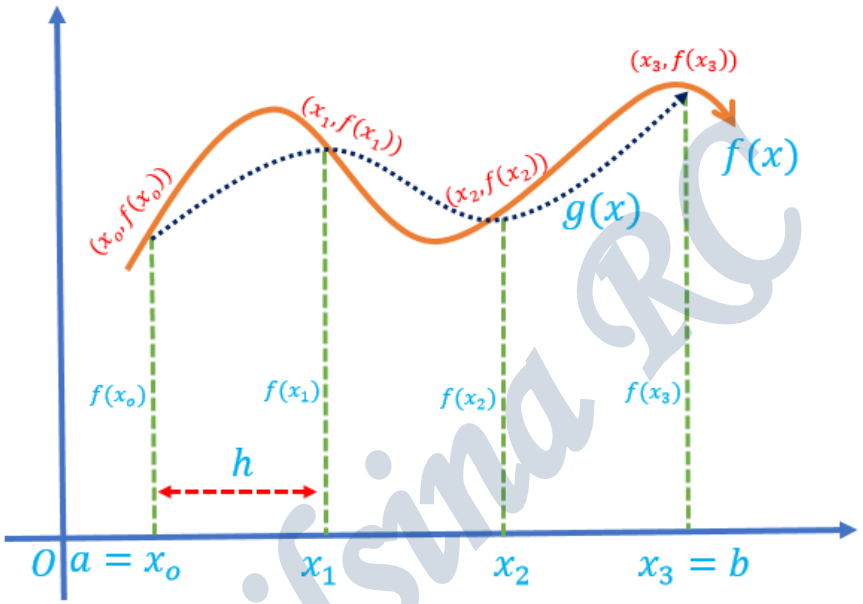
```
>> OneThird_rule
Approximated area is 9.999206e-01
```

Tamilsina RC

5.3 Simpson's 3/8-Rule

5.3.1 Single Application

Let $y = f(x)$ be a continuous curve defined on $[a, b]$. Let us partition the interval $[a, b]$ into 3 equal parts of size $h = \frac{b-a}{3}$ as shown in figure below.



Here we approximate $f(x)$ by the **cubic** polynomial $g(x)$ passing through the points $(x_o, f(x_o))$, $(x_1, f(x_1))$, $(x_2, f(x_2))$ and $(x_3, f(x_3))$ which is obtained by using **Lagrange's Interpolation Method** as follows .

$$\begin{aligned} g(x) \;=\; & \frac{(x-x_1)(x-x_2)(x-x_3)}{(x_o-x_1)(x_o-x_2)(x_o-x_3)} f(x_o) + \frac{(x-x_o)(x-x_2)(x-x_3)}{(x_1-x_o)(x_1-x_2)(x_1-x_3)} f(x_1) \\ & + \frac{(x-x_o)(x-x_1)(x-x_3)}{(x_2-x_o)(x_2-x_1)(x_2-x_3)} f(x_2) + \frac{(x-x_o)(x-x_1)(x-x_2)}{(x_3-x_o)(x_3-x_1)(x_3-x_2)} f(x_3) \end{aligned}$$

By some algebraic manipulation , we will get $g(x)$ is in the form $a_o x^3 + a_1 x^2 + a_2 x + a_3$ with $f(x_i) = g(x_i)\ ; i = 0, 1, 2, 3$ and $f(x) \approx g(x)\ \forall x \in [a, b]$. Then,

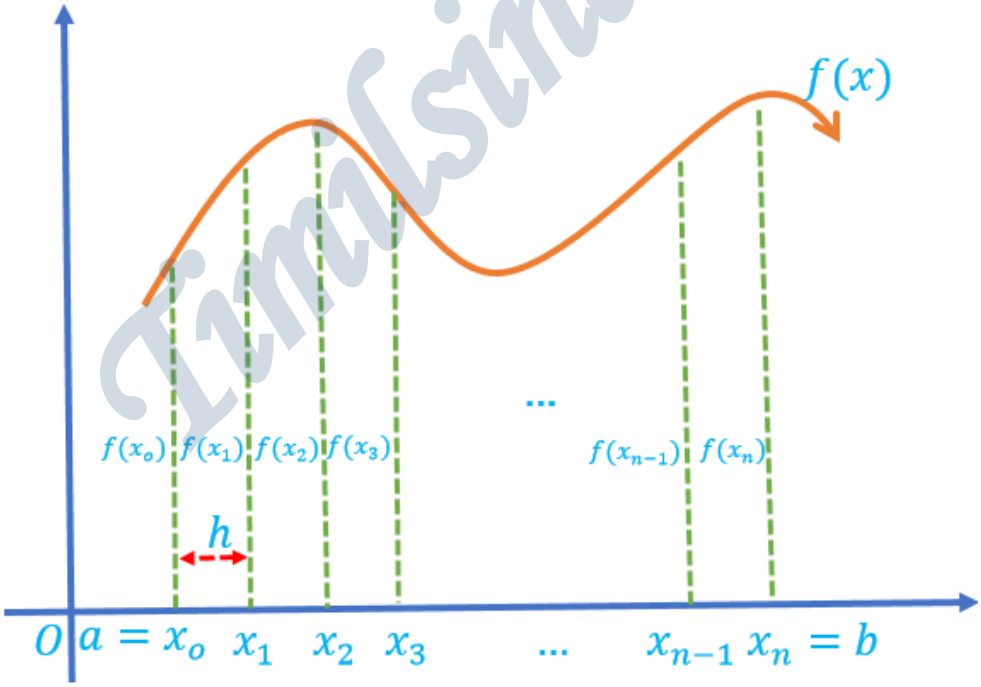
Limisina RC

$$
\begin{aligned}
I &= \int_a^b f(x)dx \\
&\approx \int_a^b g(x)dx \\
&= \frac{b-a}{8}[f(x_o) + 3f(x_1) + 3f(x_2) + f(x_3)] \\
I &\approx \frac{3h}{8}[\{f(x_o) + f(x_3)\} + 3\{f(x_1) + f(x_2)\}]
\end{aligned}
$$

is the required formula for Simpson's 3/8 rule for single application.

5.3.2 Composite Application

Let $y = f(x)$ be a continuous curve defined on $[a, b]$. Let us partition the interval $[a, b]$ into n equal parts of size h as shown in figure below.



Where, $x_1 = a + h, x_2 = a + 2h, x_3 = a + 3h, \ldots, x_n = a + nh$. Therefore, $h = \frac{b-a}{n}$, n = number of partition.

Timilsina RC

Now,

$$
\begin{aligned}
I &= \int_a^b f(x)dx \\
&= \int_{x_o=a}^{x_3} f(x)dx + \int_{x_3}^{x_6} f(x)dx + \ldots + \int_{x_{n-3}}^{x_n=b} f(x)dx \\
&\approx \frac{3h}{8}[\{f(x_o)+f(x_3)\}+3\{f(x_1)+f(x_2)\}] + \frac{3h}{8}[\{f(x_3)+f(x_6)\}+ \\
&\quad 3\{f(x_4)+f(x_5)\}] + \ldots + \frac{3h}{8}[\{f(x_{n-3})+f(x_n)\}+3\{f(x_{n-2})+f(x_{n-1})\}] \\
&\quad \because \textit{Using formula of single application.} \\
&= \frac{3h}{8}[\{f(x_o)+f(x_n)\} \\
&\quad +3\{f(x_1)+f(x_2)+f(x_4)+f(x_5)+\cdots+f(x_{n-2})+f(x_{n-1})\} \\
&\quad +2\{f(x_3)+f(x_6)+\cdots+f(x_{n-3})\}] \\
I &\approx \frac{3h}{8}[\{f(a)+f(b)\}+3\sum_{1,4,7,\ldots}^{n-2}\{f(x_i)+f(x_{i+1})\}+2\sum_{3,6,9,\ldots}^{n-3}f(x_i)]
\end{aligned}
$$

is the required formula for Simpson's 3/8 rule for composite application.

$$
\begin{aligned}
I &= \frac{3h}{8}[\{f(x_o)+f(x_3)\}+3\{f(x_1)+f(x_2)\}] \quad (\textit{Single Application}) \\
I &= \frac{3h}{8}[\{f(x_o)+f(x_n)\} \\
&\quad +3\{f(x_1)+f(x_2)+f(x_4)+f(x_5)+\cdots+f(x_{n-2})+f(x_{n-1})\} \\
&\quad +2\{f(x_3)+f(x_6)+\cdots+f(x_{n-3})\}] \ (\textit{Composite Application}) \\
&= \frac{3h}{8}[\{f(a)+f(b)\}+3\sum_{1,4,7,\ldots}^{n-2}\{f(x_i)+f(x_{i+1})\}+2\sum_{3,6,9,\ldots}^{n-3}f(x_i)]
\end{aligned}
$$

Ishwor RC

Example. 5.3:

Question: Find the value of integral $\int_0^4 xe^{2x}\,dx$ by using **Simpson's 3/8-Rule** .

Matlab Code:

Editor - D:\RC_MatLab\Three8_rule.m*

Three8_rule.m* | OneThird_rule.m

```
1  f = @(x) x*exp(2*x);
2  a = 0;        % lower limit
3  b = 4;      %upper limit
4  n = 12;       %multiple of 2
5  h = (b-a)/n ;  % step size
6  s1 = f(a)+f(b);
7  s2 = 0;
8  s3 = 0;
9  for i = 1:3:n-2
10    s2 = s2+f(a+i*h)+f(a+(i+1)*h);
11 end
12 for i = 3:3:n-3
13   s3 = s3+f(a+i*h);
14 end
15 I = 3*(h/8)*(s1+3*s2+2*s3);
16 fprintf('Approximated area is %d \n', I)
```

Command Window

New to MATLAB? See resources for Getting Started.

```
>> Three8_rule
Approximated area is 5.234900e+03
```

Imilsina RC

EXERCISE

1. Write a Matlab program for Trapezoidal rule to integrate $\int_0^{\pi} \frac{\sin x}{x}\, dx$ taking n = 6.
2. Derive formula for Composite Simpson's 3/8 rule by using single application. Write a progra to evaluate $\int_0^3 \frac{dx}{1+x}$ taking n = 6 by using Simpson's 3/8 rule.
3. Write a Matlab program for Simpson's 3/8 rule to integrate $\int_0^2 \frac{e^x}{1+x^2}\, dx$ taking h=1/3 .
4. A river is 90 m wide. The depth d is meter at a distance x meter from one bank is given by the following table

x(m)	0	10	20	30	40	50	60	70	80	90
d(m)	0	04	07	09	12	15	14	08	03	01

   Find approximately the area of the cross-section by using Simpson 3/8 rule. Also, write a MATLAB program.

5. Write short notes on Numerical Integration.

Timilsina RC

CHAPTER 6

NUMERICAL SOLUTION OF IVPS IN ODES

6.1 Forward (explicit) Euler Method

Consider an initial value problem (IVP) $y'(t) = f(t, y(t))$ with $y(t_o) = y_o$, where $f(t, y(t))$ is a smooth function. Let, y_n be the position at time t_n with step size h. By definition of derivative we have,

$$\frac{dy}{dt} = \lim_{h \to 0} \frac{y(t+h-y(t))}{h}$$
$$\frac{dy}{dt} \approx \frac{y(t+h)-y(t)}{h}$$

When $h \to 0$, it is clear that, at that time the slope at this point or tangential approximation lies completely in the curve.

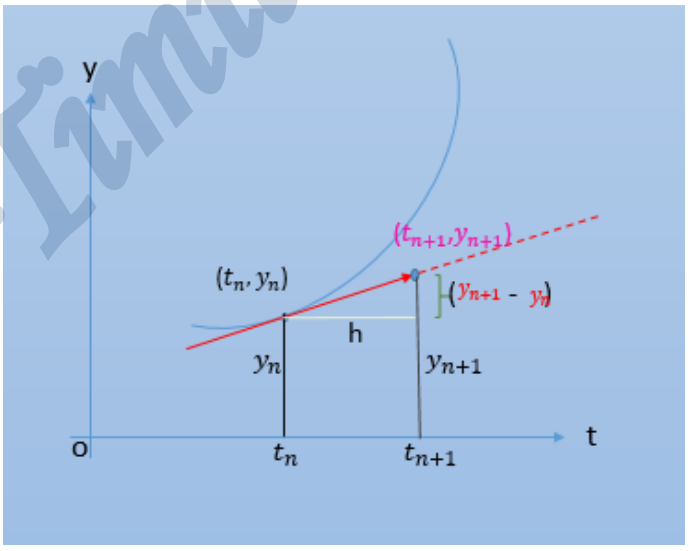


Figure 6.1: Explicit Euler Method

Slope of the curve at the point (t_n, y_n) is

$$\frac{y_{n+1} - y_n}{h} \approx \frac{dy}{dt} = f(t_n, y_n)$$

Imilsha SC

Which gives,

$$y_{n+1} = y_n + hf(t_n, y_n)$$
$$y_{n+1} = y_n + hk, \text{ where } k = f(t_n, y_n)\,.$$

Example. 6.1:

Question: Obtain a solution of a differential equation $\frac{dy}{dt} = e^{-t}$. Given the initial condition that $y(3) = 2$ and for $t = 3$ to $t = 12$ with number of steps $n = 10$.

Matlab Code:

Editor - D:\RC_MatLab\feuler.m*

Three8_rule.m | feuler.m*

```
1  f = @(t,y) exp(-t);
2  a = 3;   % initial condition
3  b = 12 ;
4  n = 10;
5  h = (b-a)/n;
6  t = a:h:b;
7  y(1) = 2;
8  for i = 1:n
9     k(i) = f(t(i),y(i));
10    y(i+1) = y(i)+h*k(i);
11 end
12 fprintf('Approximated area is %d \n', y(i+1))
```

Command Window

New to MATLAB? See resources for Getting Started.

```
>> feuler
Approximated area is 2.075498e+00
```

Tamilsina RC

Here, $f(t, y) = \frac{dy}{dt} = e^{-t}$ with $t_o = 3$, $t_n = 12$ and $y_o = 2$. Also, number of steps (n)=10. Therefore, $h = \frac{t_n - t_o}{n} = \frac{12-3}{10} = 0.9$.

Now,

t_n	$f(t_n, y_n) = e^{-t_n}\ y_{n+1} = y_n + h * e^{-t_n}$
$t_o = 3$	$y_1 = 2 + 0.9 * e^{-3} = 2.044$
$t_1 = 3.9$	$y_2 = 2.063$

$t_2 = 4.8$	$y_3 = 2.070$
$t_3 = 5.7$	$y_4 = 2.0730$
$t_4 = 6.6$	$y_5 = 2.074$
$t_5 = 7.5$	$y_6 = 2.0748$
$t_6 = 8.4$	$y_7 = 2.0749$
$t_7 = 9.3$	$y_8 = 2.0750$
$t_8 = 10.2$	$y_9 = 2.07505$
$t_9 = 11.1$	$y_{10} = 2.07506$
$t_{10} = 12$	$y_{11} = 2.07507$

Exact Solution:
Here, $\frac{dy}{dt} = e^{-t} \implies y = -e^{-t} + c.$
Using $y(3) = 2$, we will get $c = 2 + e^{-3} = 2.0497$
Hence, $y = -e^{-t} + 2.0497.$
At $t = 12, y = -e^{-12} + 2.0497 = 2.0497$

6.2 Backward (implicit) Euler Method

The backward Euler's method is similar to the forward Euler method approach except that it uses the next point t_{n+1} as the point for calculating the derivative $y'(t) = f(t, y(t))$.

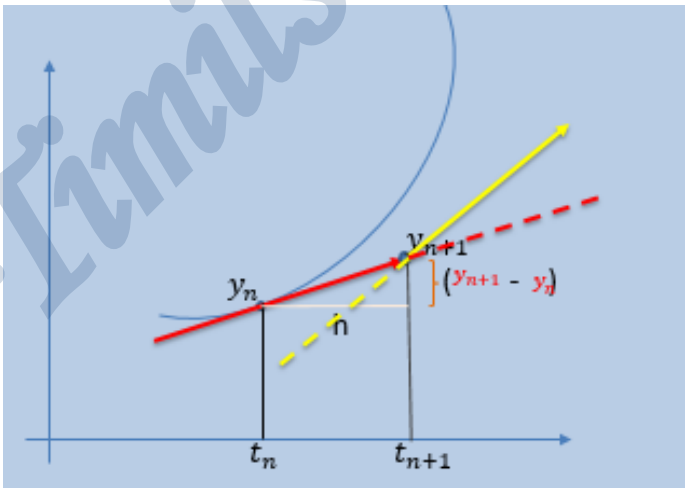


Figure 6.2: Implicit Euler Method

Here, slope at the point t_{n+1}, y_{n+1} is

$$\frac{y_{n+1} - y_n}{h} \approx \frac{dy}{dt} = f(t_{n+1}, y_{n+1})$$

Which gives,

Timilsina RC

$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$
Where $f(t_{n+1}, y_{n+1})$ is obtained from
$t_{n+1} = t_n + h$ and
$y_{n+1} = y_n + hk, \quad k = f(t_n, y_n)$.

Example. 6.2:

Question: Obtain a solution of a differential equation $\frac{dy}{dt} = e^{-t}$. Given the initial condition that $y(3) = 2$ and for $t = 3$ to $t = 12$ with number of steps $n = 10$.

Matlab Code:

Editor - D:\RC_MatLab\beuler.m*

Three8_rule.m | feuler.m

```
1 –  f = @(t,y) exp(-t);
2 –  a = 3;   % initial condition
3 –  b = 12 ;
4 –  n = 10;
5 –  h = (b-a)/n;
6 –  t = a:h:b;
7 –  y(1) = 2;
8 –  for i = 1:n
9 –     k(i) = f(t(i),y(i));
10 –    y(i+1) = y(i)+h*f(t(i)+h,y(i)+h*k(i));
11 –  end
12 –  fprintf('Approximated area is %d \n', y(i+1))
```

Command Window

New to MATLAB? See resources for Getting Started.

```
>> beuler
Approximated area is 2.030695e+00
```

Imalsina RC

6.3 2nd Order Runge-Kutta Method

This method is also known as **Improved Euler Method** or **Heun's Method**.
The Runge-Kutta 2nd order method is a numerical technique used to solve an ordinary differential equation of

the form

$$\frac{dy}{dt} = f(t, y(t)) \; ; y(t_o) = y_o$$

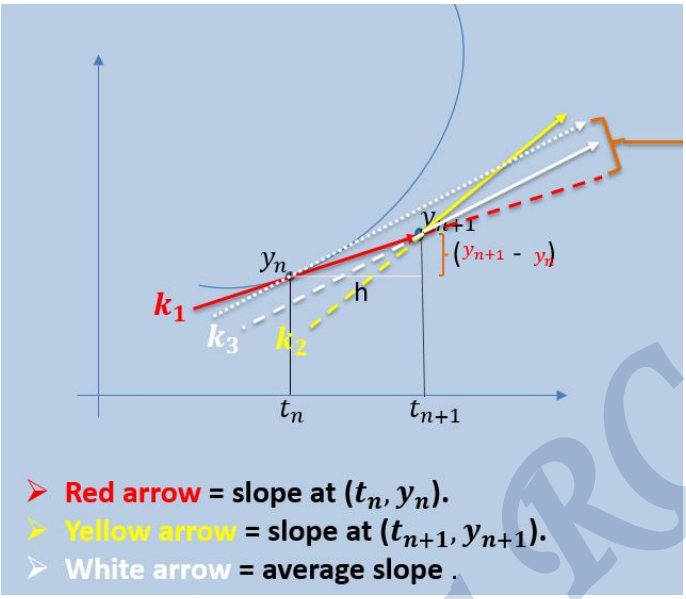


Figure 6.3: Explanation of Heun's mrthod.

From fig, it is clear that the error will minimized if we take an average slope. This concept is used to build second - order Runge - Kutta method.

Let k_1 be the slope at (t_n, y_n) i.e $k_1 = f(t_n, y_n)$ and k_2 be the slope at (t_{n+1}, y_{n+1}) i.e $k_2 = f(t_{n+1}, y_{n+1}) = f(t_n + h, y_n + hk_1)$

Thus, the average of k_1 and k_2 is given by

$$k_3 = \frac{k_1 + k_2}{2} = \frac{1}{2}[f(t_n, y_n)] + f(t_{n+1}, y_{n+1})$$

Therefore, we have

$$y_{n+1} = y_n + hk_3$$
$$y_{n+1} = y_n + \frac{h}{2}[f(t_n, y_n) + f(t_{n+1}, y_{n+1})]$$
$$y_{n+1} = y_n + \frac{h}{2}[k_1 + k_2]$$

Where,

$$k_1 = f(t_n, y_n)$$
$$k_2 = f(t_n + h, y_n + hK_1)$$

Tmilsina RC

Example. 6.3:

Question: Obtain a solution of a differential equation $\frac{dy}{dt} = e^{-t}$. Given the initial condition that $y(3) = 2$ and for $t = 3$ to $t = 12$ with number of steps $n = 10$.

Matlab Code:

Editor - D:\RC_MatLab\RK2ndmethod.m*

RK2ndmethod.m* | Untitled7*

```
f = @(t,y) exp(-t);
a = 3;    % initial condition
b = 12 ;
n = 10;
h = (b-a)/n;
t = a:h:b;
y(1) = 2;
for i = 1:n
    k(i) = f(t(i),y(i));
    k(i+1) = f(t(i)+h,y(i)+h*k(i));
    y(i+1) = y(i)+0.5*h*(k(i)+k(i+1));
end
fprintf('Approximated area is %d \n', y(i+1))
```

Command Window

New to MATLAB? See resources for Getting Started.

```
Approximated area is 2.053097e+00
```

Tanisha RC

6.4 4th Order Runge-Kutta Method

The general formula for 4th order Runge-Kutta method is shown below

$$y_{n+1} = y_n + h[\frac{K_1}{6} + \frac{K_2}{3} + \frac{K_3}{3} + \frac{K_4}{6}]$$

Where,

$K_1 = f(t_n, y_n)$

$K_2 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}K_1)$

$K_3 = f(t_n + \frac{h}{2}, y_n + \frac{h}{2}K_2)$

$K_4 = f(t_n + h, y_n + hK_3)$

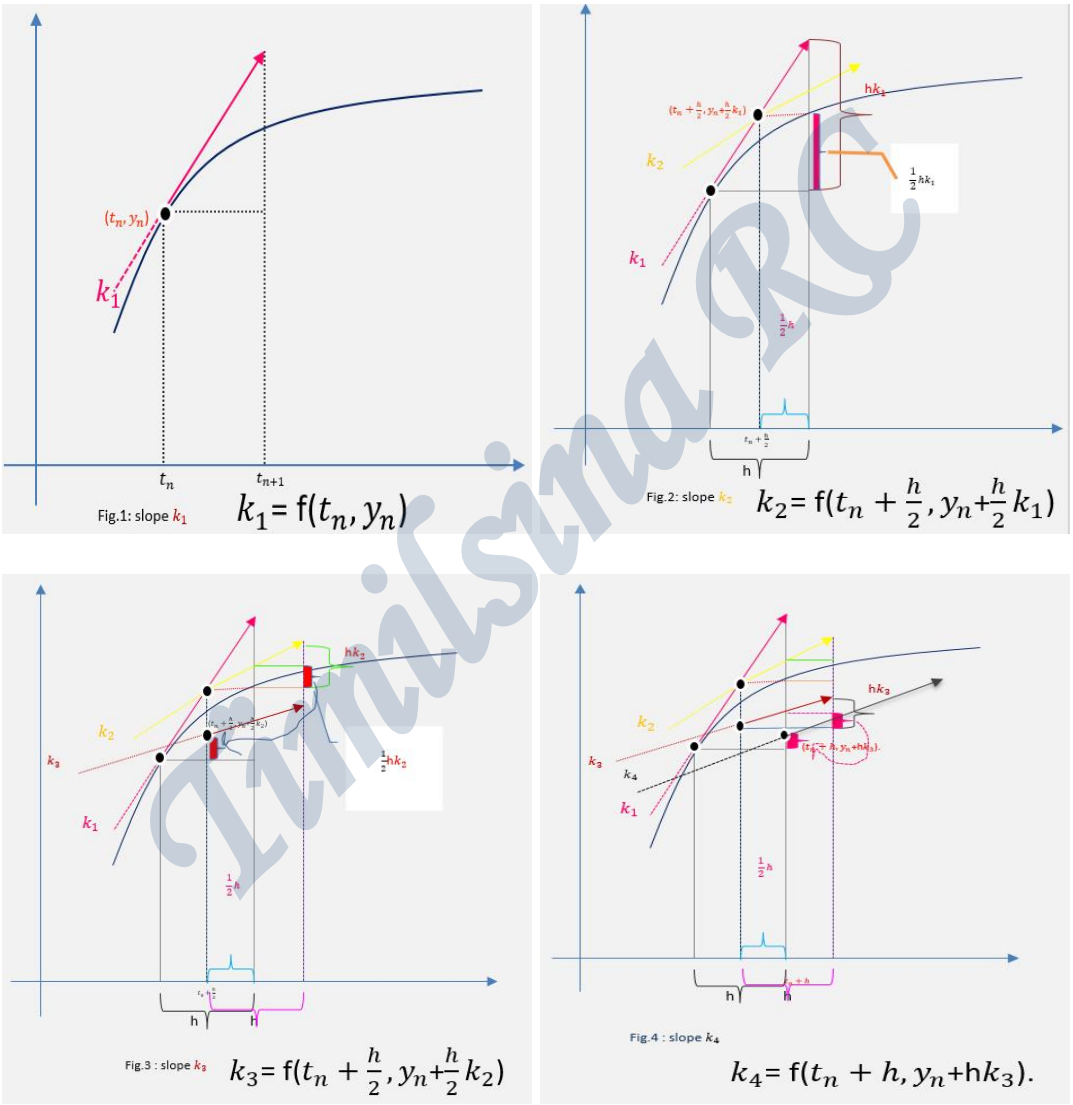


Figure 6.4: Visualization of RK4 mrthod.

Timilsina RC

Example. 6.4:

Question: Obtain a solution of a differential equation $\frac{dy}{dt} = e^{-t}$. Given the initial condition that $y(3) = 2$ and for $t = 3$ to $t = 12$ with number of steps $n = 10$.

Matlab Code:

Editor - D:\RC_MatLab\RK4thmetkod.m*

RK2ndmethod.m | RK4thmetkod.m*

```
f = @(t,y) exp(-t);
a = 3;   % initial condition
b = 12 ;
n = 10;
h = (b-a)/n;
t = a:h:b;
y(1) = 2;
for i = 1:n
    k(i) = f(t(i),y(i));
    k(i+1) = f(t(i)+0.5*h,y(i)+0.5*h*k(i));
    k(i+2) = f(t(i)+0.5*h,y(i)+0.5*h*k(i+1));
    k(i+3) = f(t(i)+h,y(i)+h*k(i+2));
    y(i+1) = y(i)+h*(k(i)+2*k(i+1)+2*k(i+2)+k(i+3))/6;
end
fprintf('Approximated area is %d \n', y(i+1))
```

Command Window

New to MATLAB? See resources for Getting Started.

```
>> RK4thmetkod
Approximated area is 2.049792e+00
```

Tanisha RC

EXERCISE

1. Integrate the first order ODE system $\frac{dx}{dt} = -2x(t) + t$ with $x(0) = 1$ using forward Euler method.
2. Write a Matlab program for Euler method to solve $\frac{dy}{dt} = e^{-t},\ y(3) = 2$ in the interval [3,12] taking number of steps, n = 10 .
3. Given IVP is $\frac{dy}{dx} = e^{-x},\ y(3) = 2$. write MATLAB code for numerical solution of given IVP by using fourth order Runge- Kutta method.
4. Write a Matlab program for Runge-Kutta fourth order method to solve $\frac{dy}{dt} = \sin(t+y) - e^{t},\ y(0) = 4$ in the interval [0,1], taking h = 0.25 .

Timilsina RC

CHAPTER 7

DEBUGGING

7.1 Debugging

Debug = De + bug, where the word 'bug' means error or problem in a program and 'de' means remove or fix it. So, debugging is a process of finding the problems or error in the codes and fix those errors.

7.2 Errors in Matlab

- **Syntatic Errors:**
  In this type of error, a Matlab program does not run or partially run and shows error. This error is caused due to the spelling error of function, misplacement of syntax and brackets, misuse of characters etc. Syntax errors clearly are a problem for novices (new amateur programmers).
  Example: If we want to find exponential of A. But, we may write B=ex(A) and Matlab shows error. Correct program is B=exp(A) .

- **Run Time Errors:**
  In this type of error a Matlab programs runs but not show what we expected. This error is also called algorithmic errors. This error may be caused due to the misplacement of operations or control and conditional statements.
  Example: Let, A=[1 2 ; 3 4] . We want to multiply A with A pair-wise. But, we may write B=A*A and we will not get desired result. The correct program is B=A.*A .

7.3 Possible Issues for Debugging

- The program is not running. (Syntatic error)
  Reasons:
  » Spelling error of functions.
  » Misplacement of syntax or brackets.

Tinkina RC

» Misuse of characters.
» Incomplete commands for ex. missing **end** in control statements.
» Incorrect m. file name it must be starts from alphabet.
» We may run function m. file instead of calling it.

- Program is partially running and showing error on a particular line. (Syntatic error)
  Reasons:
  » Required file is not present in current directory or given path.
  » Not following matrix rules: matrix inner dimensions mis match.
  » Incorrect command or statements.

- Program is running but output is incorrect. (Run-time error)
  Reason:
  » Incorrect logic for eg. to find factorial, we may use fact=fact + i instead of * sign.
  » Incorrect input data input for eg. we may enter negative value in the program of finding factorial of natural number.

7.4 Minimizing Bugs

- Write down on paper what you want to program.
- For large program, there may be multiple sections. Run one by one section particularly.Once you are satisfied with one section, move on to the next section of code simultaneously.
- Write your comments while you are coding.
- Do not make stupid mistakes in typing for eg.
  » For (i=1 to n)
  <statements>
  end

Timisina RC

EXERCISE

1. Write short notes on Debugging in Matlab.
2. What do mean by Debugging? Write possible issues for Debugging.

CHAPTER 8

NUMERICAL METHODS FOR SYSTEM OF LINEAR EQUATION

8.1 System of Linear equations

An equation of the form

$$a_{i1}x_1 + a_{i2}x_2 + \cdots + a_{in}x_n = b_i \quad ; i = 1, 2, ..., m$$

is known as system of linear equations with n variables and m linear equations. In matrix form it can be written as $AX = b$ where $A = [a_{ij}]_{m\times n}$, $X = [x_1, x_2, ..., x_n]^T$ and $b = [b_1, b_2, ..., b_m]^T$.
A system of linear equation may have one solution, no solution or infinitely many solution.

Example:

$$\begin{aligned} x + 3y - 2z &= 5 \\ 3x + 5y + z &= 7 \\ 2x + 4y + 3z &= 8 \end{aligned}$$

Where, $A = \begin{bmatrix} 1 & 3 & -2 \\ 7 & 3 & 6 \\ 8 & 4 & 3 \end{bmatrix}$, $X = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$ and $b = \begin{bmatrix} 5 \\ 7 \\ 8 \end{bmatrix}$

» X = inv(A)*b
» Ans: X = [-15, 8 , 2]

Timisina RC

8.2 Crammer's Rule

» $X_i = \frac{det(A_i)}{det_A}$ **where** A_i **is the matrix** A **with** i^{th} **column replaced by** b**. For ex.**

$$A_1 = \begin{bmatrix} 5 & 3 & -2 \\ 7 & 3 & 6 \\ 8 & 4 & 3 \end{bmatrix}$$

Example. 8.1:

Editor - D:\RC_MatLab\Cramers_Rule.m

Bisection_Method.m | Newton_Rapshon_Metho

```
1      % Cramer's Rule
2 -    A = [1  3  -2 ; 3  5  6 ; 2  4  3] ;
3 -    b = [ 5 ; 7 ; 8 ] ;
4 -    for i = 1 : length(A)
5 -        P = A ;
6 -        P( : , i) = b ;
7 -        x(i) = det(P)/det(A);
8 -    end
9 -    display(x) ;
```

» **X = [-15, 8, 2]**

Tamilsina RC

8.3 Gaussian Elimination Method

The given system can be reduced by the following process:

- The first unknown variable (x_1) in the first equation will have a non- Zero coefficient.
- the first unknown variable (x_1) may have zero coefficient in the remaining coefficient but second variable (x_2) must have non-zero coefficient in the second equation. And so on.

In Fact:

We have to reduce the give system, using elementary row operation, in the form:

$$[A:b] = \begin{bmatrix} a_{11} & a_{12} & a_{13} & \dots & a_{1n} & : b'_1 \\ 0 & a'_{22} & a'_{23} & \dots & a'_{2n} & : b'_2 \\ 0 & 0 & a'_{33} & \dots & a'_{3n} & : b'_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & : \vdots \\ 0 & 0 & 0 & \dots & a'_{mn} & : b'_m \end{bmatrix}$$

Then on solving by backward substitution method we will get the required solution as :

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n &= b'_1 \\ a'_{22}x_2 + a'_{23}x_3 + \cdots + a'_{2n}x_n &= b'_2 \\ \dots \quad \dots \quad \dots & \\ a'_{mn}x_n &= b'_m \end{aligned}$$

Example. 8.2:

```
%Gaussian elimination Method:
A = [1  3  -2; 3  5  6 ; 2  4  3];
b = [5; 7; 8];
[n,m]  =  size(A);
x = zeros(n,1);   % initialize x
for i = 1 : n-1
   m = A(i+1 : n, i) / A(i,i) ;
   A(i+1 : n , :) = A(i+1:n , :) - m*A(i ,:) ;
   b(i+1:n , :) = b (i+1:n, :) - m*b(i,:);
end
x(n , :) = b(n , :) /A(n,n);
for i = n-1 :-1 :1
   x(i,:) = (b(i,:)-A(i,i+1 : n )*x(i+1 : n , :)) / A(i,i);
end
disp(x);
```

» X = [-15, 8, 2]

Emilsina RC

8.4 Gauss Jordan Method

There is another method of solving linear simultaneous equation known as Gauss Jordan method in which the coefficients a_{ii} are made unity and all other coefficients zero.

$$[A : b] = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 & : b'_1 \\ 0 & 1 & 0 & \dots & 0 & : b'_2 \\ 0 & 0 & 1 & \dots & 0 & : b'_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots & : \vdots \\ 0 & 0 & 0 & \dots & 1 & : b'_m \end{bmatrix}$$

Then $X = [b'_1, b'_2, \dots, b'_m]$ is the solution.

Example. 8.3:

```
%Gaussian Jordan Method:
A = [1 3 -2 5;3 5 6 7;2 4 3 8];
R = rref(A) ;
X=R(:,end) ;
disp(X)
```

» X = [-15, 8, 2]

Example. 8.4:

```
%%Gauss Jordan Method
A=[13 2 3 46; 2 15 1 33; 1 -1 10 25];
[m,n]=size(A);
for j=1:m-1
        for i=j+1:n          %pivoting
        if A(j,j)==0
            t=A(j,:)
             A(j,:)=A(i,:)
            A(i,:)=t;
        end
    end
for i=j+1:m                  %converting the element below the major diagonal to zero
    A(i,:)=A(i,:)-A(j,:)*(A(i,j)/A(j,j));
end
end
for j=m:-1:2                 %convert the element above the major diagonal to zero
    for i=j-1:-1:1
        A(i,:)=A(i,:)-A(j,:)*(A(i,j)/A(j,j));
    end
end
for s=1:m                    %convert the diagonal element to ones.
    A(s,:)=A(s,:)/A(s,s);
    x(s)=A(s,n);
end
disp('gauss jordan method:');
A
x
```

» X = [2.7278, 1.6766, 2.3949]

Limisina RC

8.5 LU Factorization Method

Let A be a square matrix. An LU factorization refers to the factorization of A, with proper row and/or column orderings or permutations, into two factors : a lower triangular matrix L and an upper triangular matrix U:

$$A = LU$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ f_{12} & 1 & 0 \\ f_{31} & f_{32} & 1 \end{bmatrix} \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a'_{22} & a'_{23} \\ 0 & 0 & a''_{33} \end{bmatrix}$$

where $f_{12} = \frac{a_{12}}{a_{11}}$, $f_{31} = \frac{a_{31}}{a_{11}}$ and $f_{32} = \frac{a'_{32}}{a'_{22}}$

Example. 8.5:

```
%%LU Factorization Method
A=[12 2 3; 1 10 -2; 3 1 15];
b=[18; 16; 52];
[n,m]=size(A);
for j=1:n-1
    for i=j+1:n %pivoting
        if A(j,j)==0
            t=A(j,:)
             A(j,:)=A(i,:)
            A(i,:)=t;
        end
    end
end
for j=1:n-1
    for i=j+1:n
        m(i,j)=A(i,j)/A(j,j);
        A(i,:)=A(i,:)-m(i,j)*A(j,:);
        b(i,:)=b(i,:)-m(i,j)*b(j,:);
    end
  end
U=A
L=m
x=zeros(n,1);
x(n,:)=b(n,:)/A(n,n)
for i=n-1:-1:1
   x(i,:)=(b(i,:)-A(i,i+1:n)*x(i+1:n,:))/A(i,i);
end
x
```

» X = [0.3162, 2.2195, 3.2555]

Tamilsina RC

8.6 Gauss Seidel Method

Gauss Seidel method is an iterative method of solution of system of linear equations. It allows the user to control the round of error. We begin with the following system of linear equations in 3 variables.

$$\begin{aligned} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 &= b_1 \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 &= b_2 \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 &= b_3 \end{aligned}$$

Equation in GS form

$$x_1^{(n+1)} = \frac{1}{a_{11}}(b_1 - a_{12}x_2^{(n)} - a_{13}x_3^{(n)})$$

$$x_2^{(n+1)} = \frac{1}{a_{22}}(b_2 - a_{21}x_1^{(n+1)} - a_{23}x_3^{(n)})$$

$$x_3^{(n+1)} = \frac{1}{a_{33}}(b_3 - a_{31}x_1^{(n+1)} - a_{32}x_2^{(n+1)})$$

In General form

$$x_i^{(n+1)} = \frac{1}{a_{ii}}\left[b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(n+1)} - \sum_{j=i+1}^{k} a_{ij}x_j^{(n)}\right]$$

For all

$$n = 0, 1, 2, 3, \ldots$$

$$i = 1, 2, 3, \ldots, k$$

Example:

Given equation

$$13x + 2y + 3z = 46 \quad (2)$$

$$2x + 15y + z = 33 \quad (3)$$

$$x - y + 10z = 25 \quad (4)$$

Conditions for diagonally dominant

$$|13| > |2| + |3|$$

$$|15| > |2| + |1|$$

$$|10| > |1| + |-1|$$

Timisina RC

Step-I

$$13x = 46 - 2y - 3z$$

$$x = \frac{1}{13}(46 - 2y - 3z)$$

Similarly,

$$y = \frac{1}{15}(33 - 2x - z)$$

$$z = \frac{1}{10}(25 - x + y)$$

1st Iteration

Taking $n = 0 then, y^0 = 0, z^0 = 0,$ Then

$$x_1^1 = \frac{1}{13}(46 - 2y^0 - 3z^0) = \frac{1}{13}(46 - 2 * 0 - 3 * 0) = 3.5384$$

$$y_1^1 = \frac{1}{15}(33 - 2x^1 - z^0) = \frac{1}{15}(33 - 2 * 3.5384 - 0) = 1.7282$$

$$z_1^1 = \frac{1}{10}(25 - x^1 + y^1) = \frac{1}{10}(25 - 3.5384 + 1.7282) = 2.31902$$

2nd Iteration

$$x_2^2 = \frac{1}{13}(46 - 2y^1 - 3z^1) = \frac{1}{13}(46 - 2 * 1.7282 - 3 * 2.31902) = 2.7374$$

$$y_2^2 = \frac{1}{15}(33 - 2x^2 - z^1) = \frac{1}{15}(33 - 2 * 2.7374 - 2.31902) = 1.6804$$

$$z_1^2 = \frac{1}{10}(25 - x^2 + y^2) = \frac{1}{10}(25 - 2.7374 + 1.6804) = 2.3943$$

3rd Iteration

$$x_3^3 = \frac{1}{13}(46 - 2y^2 - 3z^2) = \frac{1}{13}(46 - 2 * 1.6804 - 3 * 2.3943) = 2.7374$$

$$y_3^3 = \frac{1}{15}(33 - 2x^3 - z^2) = \frac{1}{15}(33 - 2 * 2.7374 - 2.31902) = 1.6767$$

$$z_3^3 = \frac{1}{10}(25 - x^3 + y^3) = \frac{1}{10}(25 - 2.7374 + 1.6767) = 2.3949$$

Limlisha RC

4th Iteration

$$x_4 = \frac{1}{13}(46 - 2y_3 - 3z_3) = \frac{1}{13}(46 - 2 * 1.6767 - 3 * 2.3949) = 2.7278$$

$$y_4 = \frac{1}{15}(33 - 2x_4 - z_3) = \frac{1}{15}(33 - 2 * 2.7278 - 2.3949) = 1.6766$$

$$z_4 = \frac{1}{10}(25 - x_4 + y_4) = \frac{1}{10}(25 - 2.7278 + 1.6766) = 2.3948$$

5th Iteration

$$x_5 = \frac{1}{13}(46 - 2y_4 - 3z_4) = \frac{1}{13}(46 - 2 * 1.6766 - 3 * 2.3948) = 2.7279$$

$$y_5 = \frac{1}{15}(33 - 2x_5 - z_4) = \frac{1}{15}(33 - 2 * 2.7278 - 2.3948) = 1.6766$$

$$z_5 = \frac{1}{10}(25 - x_5 + y_5) = \frac{1}{10}(25 - 2.7278 + 1.6766) = 2.3949$$

6th Iteration

$$x_6 = \frac{1}{13}(46 - 2y_5 - 3z_5) = \frac{1}{13}(46 - 2 * 1.6766 - 3 * 2.3949) = 2.7279$$

$$y_6 = \frac{1}{15}(33 - 2x_6 - z_5) = \frac{1}{15}(33 - 2 * 2.7279 - 2.3949) = 1.6766$$

$$z_6 = \frac{1}{10}(25 - x_6 + y_6) = \frac{1}{10}(25 - 2.7279 + 1.6766) = 2.3949$$

Imalsha RC

Example. 8.6:

Editor - D:\RC_MatLab\GaussSiedel.m*

Bisection_Method.m | Newton_Rapshon_Method.m | GaussJordan.m

```
%Gauss Seidel Method:
A = [ 13  2   3 ; 2  15   1  ; 1   -1  10 ];
b = [  46 ;  33 ; 25];
p = [ 0 ;  0  ;  0 ] ;
n = 100 ;
Toll = 0.0001 ;
N = length(b) ;
x = zeros(N,1);
y = zeros(N,1);
for j = 1 : n
   for i =  1 : N
      num = b(i,end) - A(i,1:i-1)*x(1:i-1)-A(i,i+1:N)*x(i+1:N);
      x(i)  = num / A(i,i);
      p(i) = x(i) ;
   end
   fprintf("Iteration Number : %d \n", j);
   x
   if abs(x-y) < Toll
      break
   end
```

» X = [2.7279, 1.6766, 2.3949]

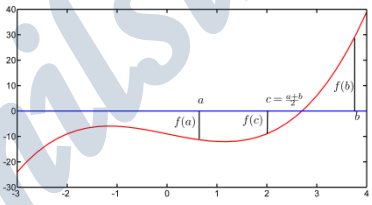
Tanilsina RC

CHAPTER 9

ROOT OF NON-LINEAR EQUATION

9.1 Bisection Method

- if $f(x)$ is continuous on $[a, b]$ with $f(a).f(b) < 0$ then its at least one root lies in (a, b). Put $c = \frac{a+b}{2}$.
- If $f(c) = 0$ then c is a root.
- If $f(c) \neq 0$ then either $f(a).f(c) < 0$ and a root lies in (a, c) or $f(b).f(c) < 0$ and a root lies in (c, b).



Example. 9.1:

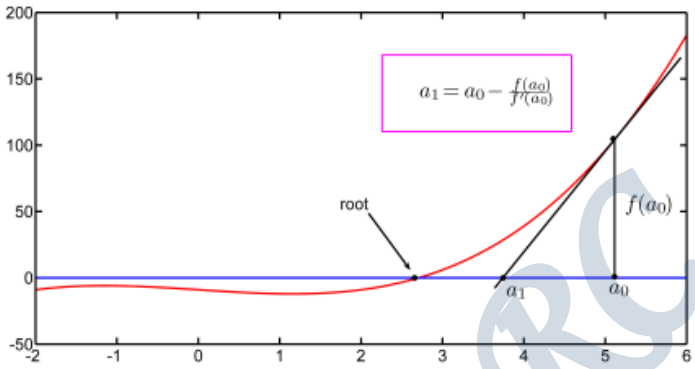
```
% Finding root by using Bisection Method:
f = @(x) x^2-26;
a = 5 ;
b = 6;
tol = 0.001;
while abs(a-b) >= tol
   x = (a+b)/2;
   if f(a)*f(x)  <  0
      b = x;
   elseif f(x) * f(b) < 0;
      a = x ;
   end
end
fprintf('Required root is : %f \n', x);
```

» Required root is : 5.098633

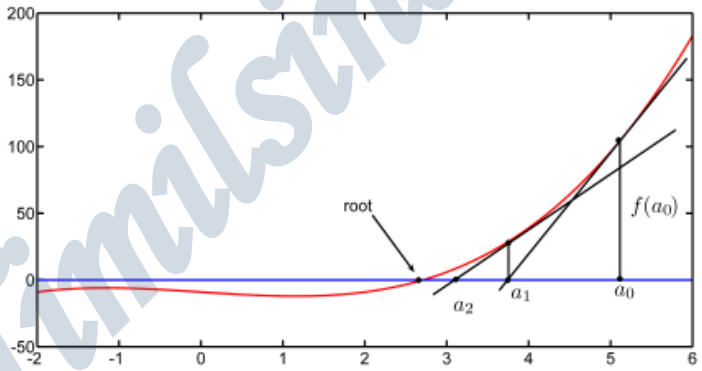
Tanilsina RC

9.2 Newton-Rapshon Method

- Let a_o be a approximate root of the equation $f(x) = 0$.
- Tangent line at $(a_o, f(a_o))$ to $y = f(x)$ is $y - f(a_o) = f'(a_o)(x - a_o)$.
- Sequence $a_o, a_1, a_2, a_3, \ldots$ generated.Where, $a_1 = a_o - \frac{f(a_o)}{f'(a_o)}$. [i.e value of x with $y = 0$ at tangent line.]



- Tangent line at $(a_1, f(a_1))$ to $y = f(x)$ is $y - f(a_1) = f'(a_1)(x - a_1)$ and $a_2 = a_1 - \frac{f(a_1)}{f'(a_1)}$.



- Tangent line at $(a_n, f(a_n))$ to $y = f(x)$ is $y - f(a_n) = f'(a_n)(x - a_n)$ and $a_{n+1} = a_n - \frac{f(a_n)}{f'(a_n)}$ for $n = 0, 1, 2, 3, \ldots$.

TamilsinaRG

Example. 9.2:

```
% Finding root by using Newton Rapshon  Method:
f = @(x) exp(-x)-4*x;
d = @(x) -exp(-x)-4;                %Derivative of f
xo= 0 ;
diff = 1 ;
tol = 0.001;
while diff >= tol
   x1 = xo - f(xo)/d(xo);
   diff = abs(x1-xo);
   xo = x1;
end
fprintf('Required root is : %f \n', xo);
```

```
Command Window
New to MATLAB? See resources for Getting Started.
>> Newton_Rapshon_Method
Required root is : 0.203888
```

EXERCISE

1. Bisection Method:

   - cos $x - xe^x = 0$ (root: 0.5178)
   - $x \sin x - 1 = 0$ root lies between $x = 1$ and $x = 1.5$. (root: 1.1133)
   - $x \log_{10} x = 1.9$ (root: 3.4961)
   - $e^x - 3x = 0$ (root: 0.6191)
   - $x^3 - 2x - 5 = 0$

2. Newton -Rapshon Method:

   - $x^4 - x - 10 = 0$
   - $x^3 - x^2 + x + 7 = 0$
   - $3x \sin x - e^x = 0$

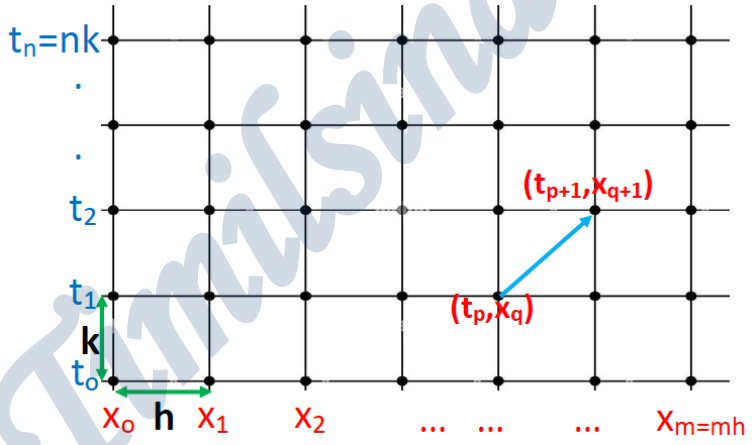
Tamilsiva RC

CHAPTER 10

NUMERICAL SOLUTION TO THE PDE

10.1 Finite Difference Method

The finite difference method is one of the techniques to obtaining the numerical solution of Partial Differential Equations (PDEs). In this method the solutions are approximated in the discrete form in the finite grid points. Increasing the number of points results increases the accuracy of the solution. Consider the grid as shown in figure below.



Tmilsina RC

Define a grid function v on the grid points (t_n, x_m), denoted by v_m^n i.e $v_m^n = v(t_n, x_m)$ is the value of v at the grid point.

10.2 Heat Equation

Heat equation: $u_t = bu_{xx} \quad ; 0 \leq x \leq L, t \geq 0$
BCs: $u(t,0) = T_1$ and $u(t,L) = T_2 \, \forall t \geq 0$
IC: $u(0,x) = f(x) \, \forall 0 \leq x \leq L$
Where b is the coefficient of diffusivity and u is a function of two independent variables space(x) and time (t).

Example 1: FTCS

$u_t = 0.05u_{xx} \quad ; 0 \leq x \leq 1, t \geq 0$
BCs: $u(t,0) = u(t,L) = 0 \, \forall t \geq 0$
IC: $u(0,x) = sin(\pi x) \, \forall 0 \leq x \leq 1$

Solution:

Given, one dimensional heat equation is $u_t = bu_{xx} \quad ; 0 \leq x \leq 1, t \geq 0$ ... (i)
Where $b = 0.05$. We can solve this problem by using finite difference method. First let us derive the forward time central space (FTCS) scheme. We have from Taylor's series Expansion:

$$\begin{aligned} u(t_n + k, x_m) &= u(t_n, x_m) + ku_t + O(k^2) \\ \Rightarrow u_t &\approx \frac{u(t_n + k, x_m) - u(t_n, x_m)}{k} \\ &= \frac{v_m^{n+1} - v_m^n}{k} \quad [\, In\ terms\ of\ grid\ function] \end{aligned}$$

And,

$$\begin{aligned} u(t_n, x_m + h) &= u(t_n, x_m) + hu_x + \frac{h^2}{2!}u_{xx} + \frac{h^3}{3!}u_{xxx} + O(h^4) \\ u(t_n, x_m - h) &= u(t_n, x_m) - hu_x + \frac{h^2}{2!}u_{xx} - \frac{h^3}{3!}u_{xxx} + O(h^4) \\ \therefore\ u(t_n, x_m + h) + u(t_n, x_m - h) &= 2u(t_n, x_m) + 2\frac{h^2}{2!}u_{xx} + O(h^4) \\ \Rightarrow u_{xx} &\approx \frac{u(t_n, x_m + h) - 2u(t_n, x_m) + u(t_n, x_m - h)}{2h} \\ &= \frac{v_{m+1}^n - 2v_m^n + v_{m-1}^n}{h^2} \end{aligned}$$

So equation (i) becomes,

$$\frac{v_m^{n+1} - v_m^n}{k} = b.\frac{v_{m+1}^n - 2v_m^n + v_{m-1}^n}{h^2}$$

$$\therefore\ v_m^{n+1} = (1 - 2b\mu)v_m^n + b\mu(v_{m+1}^n + v_{m-1}^n) \quad \text{where } \mu = \tfrac{k}{h^2}$$

Which is the required FTCS scheme. Note that the scheme is stable if $b\mu \leq \frac{1}{2}$.

Timilsina RC

Example. 10.1: FTSC

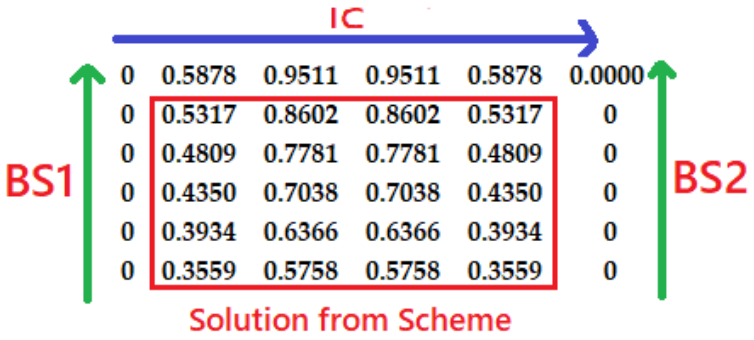
Editor - D:\RC_MatLab\HeatFTCS.m

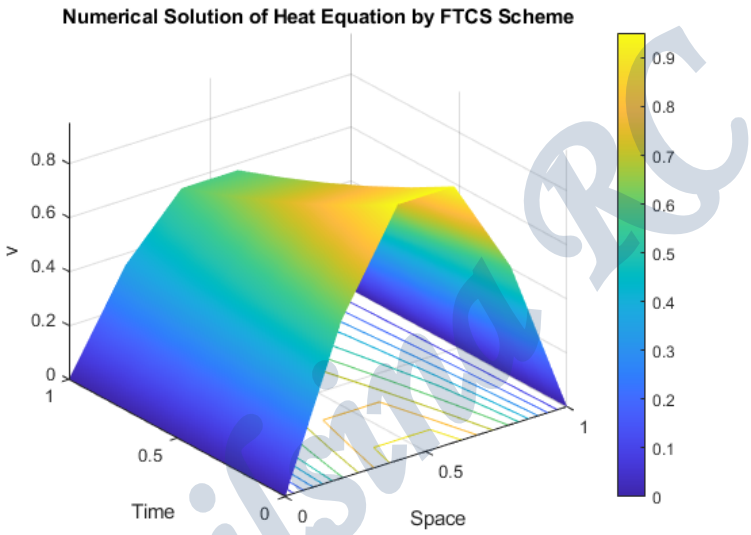
HeatFTCS.m

```
% Solution of Heat Equation by FTCS Scheme:
L = 1, T = 1 ;  %Length of rod and Time bound
b = 0.05 ;  % Thermal diffusivity
m = 5 , n = 5 ;   % No. of Space and Time Intervals
h = L/m  , k = T/n ;
c = b* k/(h*h);
if (c <= 0| c > 0.5)
   disp("The Scheme is Unstable")
else
v = zeros(n+1,m+1);   % initialize the soluion
B1 = @(t) 0;
B2 = @(t) 0 ;   % b1 and b2 are boundary conditions.
IC = @(x) sin(pi*x);  % Initial condition
for j = 2 : m+1
   v(1,j) = IC((j-1)*h);  % Assign Initial condition
end
for i = 2 : n+1
   v(i,1)  = B1((i-1)*k);
   v(i,m+1) = B2((i-1)*k) ;  % Assign boundary conditions
end
for i = 1 : n
   for j = 2 : m
      v(i+1,j) = v(i,j) + c*(v(i,j+1)-2*v(i,j)+v(i,j-1)) ; % Scheme
   end
end
disp(v)
% To plot graph
   [ p, q ] = meshgrid(0:h:L, 0:k:T) ;
   surfc(p,q,v)
   shading interp
   title('Numerical Solution of Heat Equation by FTCS Scheme')
   xlabel('Space')
   ylabel('Time')
   zlabel('v')
   colorbar ;
end
```

Timisina RC

Output:





Example 2: CTCS

We have

$$\begin{aligned} u_t &= bu_{xx} \\ \Rightarrow \quad \frac{v_m^{n+1} - v_m^{n-1}}{2k} &= b.\frac{v_{m+1}^n - 2v_m^n + v_{m-1}^n}{h^2} \\ \Rightarrow \quad v_m^{n+1} &= v_m^{n-1} + 2b\mu[v_{m+1}^n - 2v_m^n + v_{m-1}^n] \end{aligned}$$

This is the required CTCS scheme for the given heat equation.

Note:

- CTCS scheme is unstable .
- BTCS scheme is un-conditionally stable

Timisoara RC

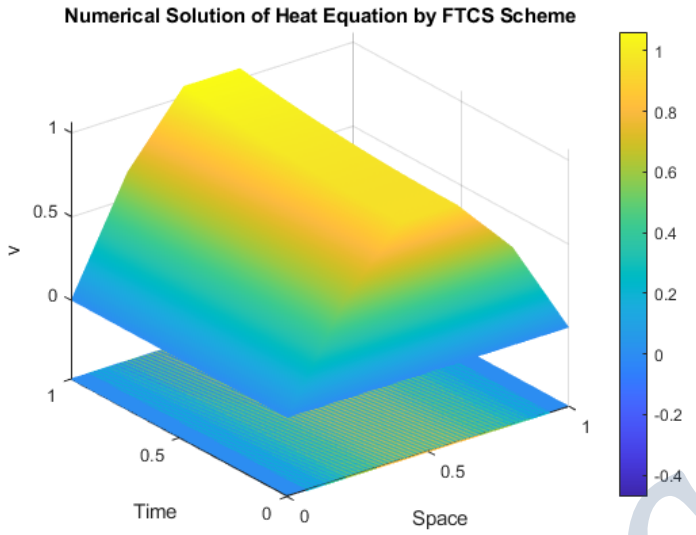
Example. 10.2: CTSC

Editor - D:\RC_MatLab\HeatCTCS.m

HeatFTCS.m | HeatCTCS.m

```
% Solution of Heat Equation by FTCS Scheme:
L = 1, T = 1 ;  %Length of rod and Time bound
b = 0.05 ;  % Thermal diffusivity
m = 5 , n = 1000 ;   % No. of Space and Time Intervals
h = L/m  , k = T/n ;
c =2* b* k/(h*h)
if (c <= 0| c > 0.01)
   disp("The Scheme is Unstable")
else
v = zeros(n+1,m+1);   % initialize the soluion
B1 = @(t) 0;
B2 = @(t) 0 ;   % b1 and b2 are boundary conditions.
IC = @(x) sin(pi*x);  % Initial condition
for j = 2 : m+1
   v(1,j) = IC((j-1)*h);  % Assign Initial condition
end
for i = 2 : n+1
   v(i,1)  = B1((i-1)*k);
   v(i,m+1) = B2((i-1)*k) ;  % Assign boundary conditions
end
for i = 2 : n
   for j = 2 : m
      v(i+1,j) = v(i-1,j) + c*(v(i,j+1)-2*v(i,j)+v(i,j-1)) ; % Scheme
   end
end
disp(v)
% To plot graph
   [ p, q ] = meshgrid(0:h:L, 0:k:T) ;
   surfc(p,q,v)
   shading interp
   title('Numerical Solution of Heat Equation by FTCS Scheme')
   xlabel('Space')
   ylabel('Time')
   zlabel('v')
   colorbar ;
end
```

Timilsina RC



Doubt: Here we can not find the value of v_m^2 from scheme because we don't have the value of v_m^0. Here matlab code assign $v_m^2 = 0, 0, 0, \ldots, 0$ due to which the value of v_m^{even} is wrong. Actually, this is unstable scheme so we could not get desired result from this scheme.

10.3 Wave Equation

The equation of the form $u_{tt} = b^2 u_{xx}$ ... (i) is known as one dimensional wave equation. In terms of grid function, this can be written as, [In fact, $u(t,x) \approx u(t_n, x_m) = v_m^n$]

$$\frac{v_m^{n+1} - 2v_m^n + v_m^{n-1}}{k^2} = b^2 . \frac{v_{m+1}^n - 2v_m^n + v_{m-1}^n}{h^2}$$

$$\Rightarrow \quad v_m^{n+1} = 2(1-\lambda)v_m^n + \lambda[v_{m+1}^n + v_{m-1}^n] - v_m^{n-1}$$

Where $\lambda = b^2 . \frac{k^2}{h^2}$. This is the **CTCS** scheme for the given wave equation (i). Note that the scheme is stable if $\lambda \leq 1$.

Example:

$u_{tt} = 1u_{xx}\ 0 \leq x \leq 1, t \geq 0$

BCs: $u(t,0) = u(t,1) = 0 \quad ; t \geq 0$

ICs: $u(0,x) = 10 \sin \pi x$ and $u_t(0,x) = 0 \quad ; 0 \leq x \leq 1$

Solution:

Formula to assign the IC $u_t(0,x) = 0$:

From above scheme, Put $n = 0$;

$$v_m^1 = 2(1-\lambda)v_m^0 + \lambda[v_{m+1}^0 + v_{m-1}^0] - v_m^{-1} \quad \ldots(i)$$

Timilsina RC

Also, we have for central time ,

$$\begin{aligned} u_t &= \frac{v_m^{n+1} - v_m^{n-1}}{2k} \\ 2ku_t &= v_m^{n+1} - v_m^{n-1} \\ 2ku_t &= v_m^1 - v_m^{-1} \quad for\ n = 0 \\ v_m^{-1} &= v_m^1 - 2ku_t \end{aligned}$$

So from (i)

$$\begin{aligned} v_m^1 &= 2(1-\lambda)v_m^0 + \lambda[v_{m+1}^0 + v_{m-1}^0] - [v_m^1 - 2ku_t] \\ \therefore\ v_m^1 &= (1-\lambda)v_m^0 + 0.5\lambda[v_{m+1}^0 + v_{m-1}^0] + 2ku_t \end{aligned}$$

In this example, $u_t = u_t(t,x) \approx u_t(t_n, x_m) = 0$ (i.e IC).

Timilsina RC

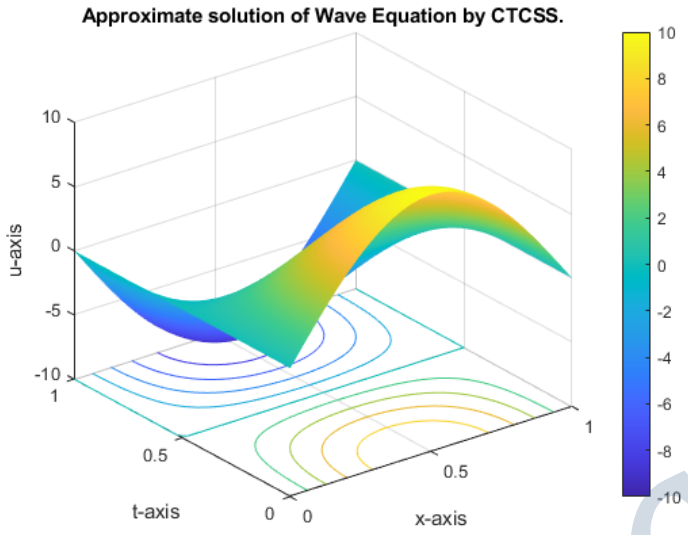
Example. 10.3: Wave Equation

Editor - D:\RC_MatLab\Wave_CTCS.m

HeatFTCS.m | HeatCTCS.m | Wave_CTCS.m

```
% Numerical solution of Wave Equation by using FDM
L = 1;        % spatial length
T = 1;        % upper time bound
b = 1;        % constat a in u_tt = a u_xx
m = 50;       % no. of intervals in space
n = 50;       % no. of intervals in time
h = L/m;
k = T/n;
c = b*(k/h)^2;
if (c > 1 | c <= 0)
   disp('The CTCSS is unstable.');
else
   v = zeros(n+1,m+1);
   B1 = @(t) 0;       %first boundary condition
   B2 = @(t) 0;       %second boundary condition
   IC1 = @(x) 10*sin(pi*x);       %first initial condition
   IC2 = @(x) 0;              %second initial condition
   for j = 1 : m+1
      v(1,j) = IC1((j-1)*h);
   end
   for i = 2 : n+1
      v(i,1) = B1((i-1)*k);
      v(i,m+1) = B2((i-1)*k);
   end
   for j = 2 : m
      v(2,j) = (1-c)*v(1,j) + 0.5*c*(v(1,j+1)+v(1,j-1)) + k*IC2((j-1)*h);
   end
   for i = 2 : n
      for j = 2  : m
         v(i+1,j)= 2*(1-c)*v(i,j) + c*(v(i,j+1)+v(i,j-1)) - v(i-1,j);
      end
   end
   [p q] = meshgrid(0:h:L,0:k:T);
   surfc(p,q,v,'Edgecolor','none');
   shading interp;
   title('Approximate solution of Wave Equation by CTCSS.');
   xlabel('x-axis');
   ylabel('t-axis');
   zlabel('u-axis');
   colorbar;
   flipud(v)
end
```

Tanilsina RC



10.4 Laplace Equation

The equation of the form $u_{xx} + u_{yy} = 0$ ... (i) is known as Laplace equation. Note that it has no time parameter. In terms of grid function this can be written as

$$\frac{v_{m+1,l} - 2v_{m,l} + v_{m-1,l}}{h_1^2} + \frac{v_{m,l+1} - 2v_{m,l} + v_{m,l-1}}{h_2^2} = 0$$

$$Where,\ \Delta x = h_1, \Delta y = h_2$$

$$h_2^2[v_{m+1,l} + v_{m-1,l}] + h_1^2[v_{m,l+1} + v_{m,l-1}] - 2(h_1^2 + h_2^2)v_{m,l} = 0$$

Therefore,

$$v_{m,l} = \frac{h_2^2[v_{m+1,l} + v_{m-1,l}] + h_1^2[v_{m,l+1} + v_{m,l-1}]}{2(h_1^2 + h_2^2)}$$

This is the required scheme for the given Laplace equation (i) which is un-conditionally stable.

Example:

$u_{xx} + u_{yy} = 0 \quad 0 \le x \le 2, 0 \le y \le 2$

BCs: $u(0, y) = 0, u(2, y) = 5$

and $u(x, 0) = 0, u(x, 2) = -2$

Solution:

Timilsina RC

Example. 10.4: Laplace Equation

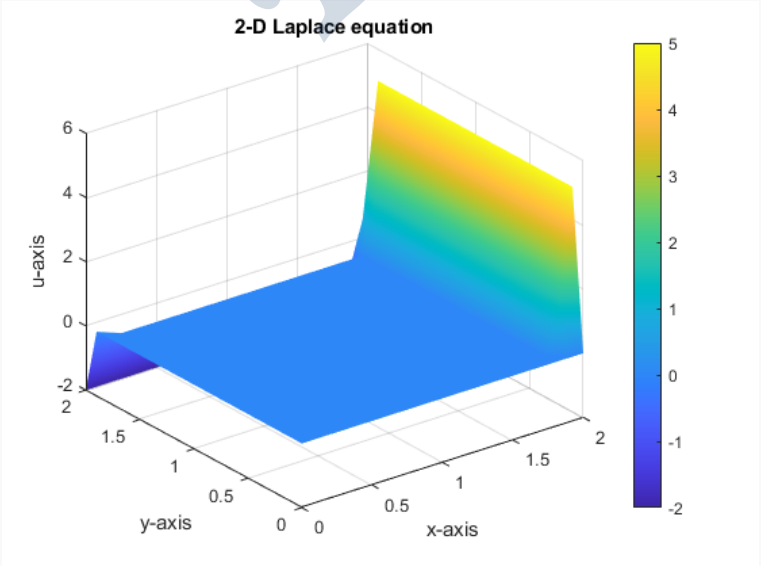
Editor - D:\RC_MatLab\Laplace.m

Laplace.m

```
% Laplace Equation
Lx = 2;              %length along x-axis
Ly = 2;              %length along y -axis
m=20;                %no. of x-intervals
n=20;                %no. of y-intervals
h=Lx/m;              %step size along x-axis
k=Ly/n;              %step size along y-axis
v = zeros(n+1,m+1);
%Boundary conditions
v(:,1)=0;
v(:,m+1)=5;
v(1,:)=0;        %Neumann condition
v(n+1,:)= -2;        %Neumann condition
for i=2:m;
for j=2:n;
  v(i,j)=((k^2*(v(i+1,j)+v(i-1,j)))+(h^2*(v(i,j+1)+v(i,j-1))))/(2*(h^2+k^2));
end
end
%Plotting the solution
[ p, q] =meshgrid(0:h:Lx,0:k:Ly)
surf(p,q,v);
shading interp
title('2-D Laplace equation');
xlabel('x-axis');
ylabel('y-axis');
zlabel('u-axis');
```

Tamilsina RC

Output:

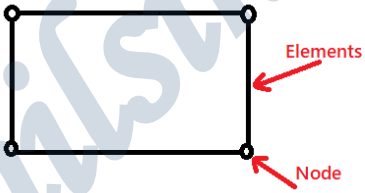


10.5 Finite Element Method

The finite element method (FEM) is mathematical numerical technique to find the approximate solution of partial differential equations.
FEM is the most powerful method for the numerical solution of initial - boundary value problem characterized by PDEs.
FEM can be use to solve the engineering and mathematical physics that like structural analysis, heat transfer, fluid flows, mass transport and electromagnetic potential.
The method yields approximate values of the unknown at discrete number of points. To solve the problem it sub-divides a large problem into simpler points that are call finite elements. The simple equation that model these finite elements are then assembled into a larger system of equations that models the entire problem.

Elements:
When a domain (a geometrical region) is meshed, it decomposes into a series of discrete (here finite) elements. The elements in the mesh should confirm very well to the geometry and the representation should be a good approximation of the geometry.

Note:
Adjacent elements are connected to each other at the node. A node is simply a point in space define by its co-ordinates.



- One dimensional element: Bar-element.
- Two dimensional element: Triangular ,quadrilateral element.
- Three dimensional element: Tetrahedral (Pyramid), Hexahedral (cube) element.

Timilsina KC

BIBLIOGRAPHY

[1] H.David (2005), **Introduction to Matlab for Engineering Students**, Northwestern University.

[2] K.Peter (2008), **MATLAB for Beginners: A Gental Approach**, Petra Books.

Timilsina RC